

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA HÀ NỘI
KHOA TOÁN – CƠ – TIN HỌC



BÁO CÁO BÀI TẬP LỚN
Môn học: Computer Vision

**Phân loại món ăn trên dataset Food-101 từ hướng cổ
diễn (HOG + Color Histogram + SVM)
đến phân loại bằng CNN và ứng dụng vào ước lượng
khẩu phần ăn**

Giảng viên hướng dẫn:

TS. Cao Văn Chung

Nhóm thực hiện:

Hoàng Minh Quang – 23001917

Nguyễn Quang Minh – 23001906

Nguyễn Thị Quỳnh Anh – 23001496

Mục lục

MỞ ĐẦU	1
1 CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP	2
1.1 Cơ sở lý thuyết	2
1.1.1 Biểu diễn ảnh số	2
1.1.2 Các thao tác trên điểm ảnh	3
1.1.3 Các thao tác hình thái	3
1.1.4 Gradient ảnh và bộ mô tả HOG	3
1.1.5 Color Histogram trên không gian HSV	4
1.1.6 Support Vector Machine (SVM)	5
1.1.7 Transfer Learning	5
1.1.8 YOLO-World và Segment Anything Model (SAM)	7
1.2 Dữ liệu và phương pháp xử lý	8
1.2.1 Bộ dữ liệu Food-101	8
1.2.2 Lựa chọn class thực nghiệm	9
1.2.3 Phân chia tập dữ liệu	10
1.2.4 Phân tích phân bố màu theo class	10
1.2.5 Tiền xử lý ảnh	12
1.2.6 Phân tích biên và hình thái	12
1.2.7 Trích xuất đặc trưng	13
1.2.8 Tổng quan pipeline và các hướng so sánh	14
2 THỰC NGHIỆM VÀ ĐÁNH GIÁ – PHƯƠNG PHÁP CỔ ĐIỂN	15
2.1 Thiết lập thực nghiệm	15
2.1.1 Cấu hình SVM	15
2.1.2 Các kịch bản thực nghiệm và chỉ số đánh giá	15
2.2 Thực nghiệm	16
2.2.1 Tổng quan kết quả	16

2.2.2	So sánh chi tiết: Hướng 1 vs. Hướng 2 trên 15 class	16
2.2.3	Kết quả: HOG + Color Histogram + SVM trên 24 class	18
2.3	Phân tích và thảo luận	19
2.3.1	Phân tích nguyên nhân cải thiện khi bổ sung Color Histogram	19
2.3.2	Phân tích kết quả 24 class	20
2.3.3	Thảo luận về giới hạn của phương pháp cổ điển	20
3	HỌC SÂU, ƯỚC LƯỢNG KHẨU PHẦN VÀ ỨNG DỤNG	21
3.1	Thiết lập thực nghiệm học sâu	21
3.1.1	Môi trường và cấu hình phần cứng	21
3.1.2	Cấu hình huấn luyện EfficientNet-B0	21
3.1.3	Thực nghiệm Transfer Learning	22
3.1.4	Thực nghiệm EfficientNet-B0 trên SAM Crops (Segmentation-First)	24
3.2	So sánh tổng hợp bốn phương pháp	27
3.3	Ước lượng khẩu phần ăn	28
3.3.1	Phạm vi và giới hạn bài toán	28
3.3.2	Pipeline ước lượng khẩu phần	28
3.3.3	Phân tích giới hạn và hướng phát triển cho ước lượng khẩu phần	34
3.4	Xây dựng ứng dụng giao diện đồ họa (GUI)	34
3.4.1	Tổng quan kiến trúc ứng dụng	34
3.4.2	Mô tả giao diện	34
3.4.3	Triển khai và demo	35
	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	36

Danh sách hình vẽ

1.1	Một số hình ảnh minh họa từ bộ dữ liệu Food-101	8
1.2	So sánh phân bố màu RGB/HSV/Lab: pizza, sushi và ice_cream	11
1.3	Trực quan hóa tiền xử lý và phân đoạn ảnh thức ăn	13
1.4	So sánh HOG visualization: class có accuracy cao vs. thấp	13
1.5	Pipeline Hướng 1: HOG + SVM	14
1.6	Pipeline Hướng 2: HOG + Color Histogram + SVM	14
1.7	Pipeline Hướng 3: Transfer Learning EfficientNet-B0 trên ảnh gốc	14
1.8	Pipeline Hướng 4: Segmentation-First với YOLO-World + SAM	14
2.1	So sánh accuracy từng class giữa hai hướng tiếp cận	16
2.2	So sánh Confusion Matrix: Hướng 1 vs. Hướng 2 (split 70/30)	17
2.3	Accuracy từng class – 24 class, Split 80/20	18
2.4	Accuracy từng class – 24 class, Split 70/30	18
3.1	Confusion Matrix trước và sau khi fine-tune	23
3.2	Minh họa kết quả pipeline: Original - YOLO BBox Overlay - SAM Crop	25
3.3	Tổng hợp kết quả giữa B1 và B2	25
3.4	Kết quả phân loại	26
3.5	Confusion Matrix của hệ thống phân loại khẩu phần	30
3.6	Độ chính xác theo từng lớp món ăn (15 classes)	30
3.7	Phân bố điểm dự đoán khẩu phần	31
3.8	Một số kết quả dự đoán khẩu phần của hệ thống	33
3.9	Kiến trúc pipeline tích hợp trong ứng dụng GUI	34
3.10	Giao diện demo hệ thống nhận diện và ước lượng khẩu phần món ăn	35
3.11	Kết quả dự đoán món ăn và thông tin ước lượng	35

Danh sách bảng

1.1	Siêu tham số HOG	4
1.2	Cấu trúc vector đặc trưng kết hợp (Hướng 2)	5
1.3	Kiến trúc EfficientNet-B0. SE ratio = 0.25; tất cả Conv đi kèm BN + SiLU.	7
1.4	Danh sách 15 class chính và đặc điểm phân loại	9
1.5	9 class bổ sung (24 class)	10
1.6	Số lượng mẫu theo tập dữ liệu và tỷ lệ chia	10
2.1	Siêu tham số SVM – dùng thống nhất cho cả hai hướng cổ điển	15
2.2	Kết quả tổng quan ba hướng thực nghiệm cổ điển	16
2.3	Accuracy từng class: Hướng 1 vs. Hướng 2 – split 70/30	17
2.4	Accuracy từng class – HOG + Color Histogram + SVM, 24 class	19
3.1	Cấu hình phần cứng và môi trường thực nghiệm học sâu	21
3.2	Cấu hình huấn luyện EfficientNet-B0	21
3.3	Kết quả classification report của mô hình CNN sau fine-tune	23
3.4	Kết quả định lượng YOLO-World và SAM	24
3.5	Bảng so sánh tổng hợp bốn phương pháp (15 class)	27
3.6	Năm đặc trưng hình học dùng để ước lượng khẩu phần	29

MỞ ĐẦU

Nhận diện món ăn tự động và ước lượng khẩu phần thô là bài toán có nhiều ứng dụng thực tiễn như hỗ trợ theo dõi dinh dưỡng cá nhân, gợi ý chế độ ăn, tự động hóa thanh toán trong nhà hàng và phân tích thói quen ăn uống. Tuy nhiên, bài toán này vẫn còn nhiều thách thức do sự đa dạng về hình dạng, màu sắc, góc chụp, điều kiện ánh sáng và cách trình bày của cùng một món ăn.

Đề tài tập trung giải quyết hai nhiệm vụ là phân loại món ăn và ước lượng khẩu phần ăn trong ảnh thành ba mức Small / Medium / Large trên tập con 15 lớp được chọn từ bộ dữ liệu chuẩn Food-101. Tập dữ liệu với 1.000 ảnh mỗi lớp bao gồm các lớp: pizza, hamburger, french_fries, ice_cream, chocolate_cake, sushi, ramen, fried_rice, omelette, pancakes, hot_dog, grilled_salmon, caesar_salad, donuts, dumplings. Điểm đặc biệt là bộ dữ liệu không có ground truth về segmentation hay kích thước khẩu phần thực tế, chỉ có nhãn lớp, do đó việc ước lượng khẩu phần chỉ ở mức thô dựa trên tỷ lệ diện tích vùng thức ăn.

Để giải quyết bài toán phân loại, nhóm thực hiện so sánh hai nhóm tiếp cận chính. **Nhóm phương pháp cổ điển** (thực hiện ở phần giữa kỳ) sử dụng đặc trưng thủ công gồm Histogram of Oriented Gradients (HOG) kết hợp Color Histogram trên không gian HSV, sau đó phân loại bằng SVM với kernel RBF. Hướng này cho thấy việc bổ sung thông tin màu sắc giúp cải thiện đáng kể so với chỉ dùng HOG. **Nhóm phương pháp học sâu** (thực hiện ở phần cuối kỳ) áp dụng Transfer Learning với mạng EfficientNet-B0 và đề xuất thêm pipeline Segmentation-First sử dụng YOLO-World kết hợp Segment Anything Model (SAM) để tách vùng thức ăn trước khi phân loại.

Ngoài ra, nhóm xây dựng một pipeline ước lượng khẩu phần dựa trên các đặc trưng hình học trích xuất từ mask phân đoạn và triển khai toàn bộ hệ thống dưới dạng ứng dụng có giao diện đồ họa (GUI).

Báo cáo được tổ chức thành ba chương: Chương 1 trình bày cơ sở lý thuyết và phương pháp; Chương 2 trình bày thực nghiệm và đánh giá các phương pháp cổ điển; Chương 3 trình bày thực nghiệm học sâu, ước lượng khẩu phần và giao diện ứng dụng.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP

1.1 Cơ sở lý thuyết

1.1.1 Biểu diễn ảnh số

Ảnh số và không gian màu

Một ảnh số màu được biểu diễn dưới dạng ma trận ba chiều $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$, trong đó H , W lần lượt là chiều cao và chiều rộng tính bằng pixel, và C là số kênh màu.

Không gian màu RGB

Không gian màu RGB biểu diễn ảnh bằng ba kênh Red, Green và Blue. Trong đó mỗi pixel được mô tả bởi bộ ba giá trị (R, G, B) , thường nằm trong khoảng $[0, 255]$ đối với ảnh 8-bit. Trong đề tài, ảnh được đọc bằng OpenCV ở định dạng BGR, sau đó chuyển sang RGB khi cần hiển thị bằng Matplotlib.

Không gian màu Grayscale

Chuyển đổi từ RGB sang Grayscale được thực hiện theo công thức chuẩn ITU-R BT.601:

$$Y = 0.299R + 0.587G + 0.114B \quad (1.1)$$

Trong pipeline HOG+SVM (Hướng 1), bước chuyển đổi này là bắt buộc trước khi tính HOG vì HOG hoạt động trên gradient cường độ điểm ảnh đơn kênh. Đây cũng là nguồn gốc của hạn chế lớn nhất của Hướng 1: toàn bộ thông tin màu sắc bị loại bỏ. Hướng 2 khắc phục điểm này bằng cách bổ sung Color Histogram được tính trên ảnh màu trước bước chuyển grayscale.

Không gian màu HSV

Không gian màu HSV gồm ba thành phần:

- **Hue (H):** Sắc độ màu, giá trị $[0^\circ, 360^\circ]$ (OpenCV dùng $[0, 180]$ để H vừa trong uint8).
- **Saturation (S):** Độ bão hòa, $[0, 255]$.
- **Value (V):** Độ sáng, $[0, 255]$.

So với RGB, HSV tách thông tin màu sắc khỏi cường độ sáng rõ ràng hơn, giúp kênh H ổn định hơn khi điều kiện ánh sáng thay đổi.

Không gian màu Lab

Không gian màu Lab tách riêng kênh độ sáng L và hai kênh màu a (xanh lá–đỏ), b (xanh dương–vàng). Đặc điểm này phù hợp cho bước tăng cường tương phản: chỉ cần xử lý kênh L mà không ảnh hưởng đến thông tin màu.

1.1.2 Các thao tác trên điểm ảnh

Chuẩn hóa cường độ đưa giá trị pixel từ khoảng $[0, 255]$ về $[0, 1]$:

$$I_{\text{norm}} = \frac{I}{255} \quad (1.2)$$

Phân tích histogram được sử dụng để quan sát phân bố giá trị pixel trên từng kênh màu RGB, HSV và Lab.

Tăng cường tương phản được thực hiện bằng CLAHE trên kênh L của không gian Lab.

1.1.3 Các thao tác hình thái

Trong pipeline hiện tại, nhóm sử dụng các phép biến đổi hình thái như Closing, Opening, Erosion và Dilation kết hợp với phát hiện biên Canny và ngưỡng màu Otsu trên không gian HSV và Lab để trích xuất vùng thức ăn. Việc phân tích mật độ cạnh trên vùng thức ăn đã được tách ra giúp gián tiếp dự đoán hiệu năng của bộ mô tả HOG.

1.1.4 Gradient ảnh và bộ mô tả HOG

Gradient của ảnh

Gradient tại điểm (x, y) được tính bằng bộ lọc Sobel:

$$G_x = I * K_x, \quad G_y = I * K_y \quad (1.3)$$

với K_x, K_y là kernel Sobel theo phương ngang và dọc. Biên độ và hướng gradient:

$$|G| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan\left(\frac{G_y}{G_x}\right) \quad (1.4)$$

Histogram of Oriented Gradients (HOG)

HOG được đề xuất bởi Dalal và Triggs (2005) [1] và hoạt động theo các bước:

1. **Chuẩn hóa gamma/màu:** Giảm ảnh hưởng của điều kiện ánh sáng.
2. **Tính gradient:** Áp dụng bộ lọc $[-1, 0, 1]$ theo cả hai chiều.
3. **Xây dựng histogram:** Ảnh được chia thành các cell (8×8 pixel). Trong mỗi cell, các gradient được lượng tử hóa thành 9 bins theo hướng (0° – 180°).
4. **Chuẩn hóa block:** Các cell được nhóm thành block (2×2 cell), vector đặc trưng trong mỗi block được chuẩn hóa L2-Hys.
5. **Nối vector:** Tất cả block được nối thành vector đặc trưng cuối cùng.

Với ảnh 128×128 , kích thước cell 8×8 , block 2×2 cell và 9 bins hướng, vector HOG có chiều dài $225 \times 36 = 8100$ chiều.

Bảng 1.1: Siêu tham số HOG

Tham số	Giá trị
Kích thước ảnh đầu vào	128×128
Pixels per cell	8×8
Cells per block	2×2
Số bins hướng	9
Block normalization	L2-Hys
Chiều dài vector đặc trưng	8.100

1.1.5 Color Histogram trên không gian HSV

Nguyên lý Color Histogram

$$h_k(b) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[I_k(i) \in \text{bin}_b], \quad k \in \{H, S, V\} \quad (1.5)$$

Histogram được chuẩn hóa L1 về tổng bằng 1 để bất biến với kích thước ảnh.

Cấu hình Color Histogram trong đề tài

- Kênh H: 16 bins (phạm vi $[0, 180]$)
- Kênh S: 8 bins (phạm vi $[0, 256]$)

- Kênh V: 8 bins (phạm vi $[0, 256]$)

Tổng cộng: $16 + 8 + 8 = 32$ chiều.

Kết hợp HOG và Color Histogram

$$\mathbf{f}_{\text{combined}} = [\mathbf{f}_{\text{HOG}} \parallel \mathbf{f}_{\text{Color}}] \in \mathbb{R}^{8132} \quad (1.6)$$

Bảng 1.2: Cấu trúc vector đặc trưng kết hợp (Hướng 2)

Thành phần	Số chiều	Ý nghĩa
HOG	8.100	Hình dạng, cạnh, cấu trúc gradient
Color Histogram H	16	Phân bố sắc độ
Color Histogram S	8	Phân bố độ bão hòa
Color Histogram V	8	Phân bố độ sáng
Tổng	8.132	Vector đặc trưng cuối cùng

1.1.6 Support Vector Machine (SVM)

Nguyên lý SVM nhị phân

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \forall i \quad (1.7)$$

SVM đa lớp

Để phân loại 15 class, chiến lược One-vs-One (OvO) được áp dụng, tạo ra $\binom{15}{2} = 105$ bộ phân loại nhị phân.

Kernel RBF

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (1.8)$$

1.1.7 Transfer Learning

Nguyên lý Transfer Learning

Transfer Learning [13] là chiến lược huấn luyện mô hình học sâu dựa trên nguyên lý tái sử dụng tri thức: một mô hình đã được huấn luyện trên bài toán nguồn quy mô lớn được dùng làm

điểm khởi đầu thay vì khởi tạo ngẫu nhiên cho bài toán đích. Nền tảng lý thuyết nằm ở tính chất phân cấp của các đặc trưng CNN: các lớp đầu học các đặc trưng tổng quát có thể tái sử dụng xuyên suốt các miền ảnh khác nhau, trong khi chỉ các lớp sâu mới mang tính đặc thù cho từng bài toán cụ thể [14].

Kiến trúc EfficientNet-B0

Compound Scaling. EfficientNet [7] được xây dựng trên nguyên lý Compound Scaling: thay vì mở rộng mô hình theo một chiều đơn lẻ, EfficientNet cân bằng đồng thời ba chiều với hệ số tỉ lệ kép ϕ :

$$\text{depth: } d = \alpha^\phi, \quad \text{width: } w = \beta^\phi, \quad \text{resolution: } r = \gamma^\phi \quad (1.9)$$

với ràng buộc $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ ($\alpha, \beta, \gamma \geq 1$) để tổng FLOPS tăng xấp xỉ 2^ϕ lần. Các hằng số $\alpha = 1.2, \beta = 1.1, \gamma = 1.15$ được tìm bởi neural architecture search trên EfficientNet-B0 [7]. Phương pháp này đảm bảo rằng khi scale up, các tài nguyên tính toán bổ sung được phân bổ tối ưu thay vì lãng phí vào một chiều duy nhất.

Mobile Inverted Bottleneck. Khối xây dựng cơ bản của EfficientNet là MBConv, kế thừa từ MobileNetV2 [17]. MBConv thực hiện chuỗi biến đổi:

1. **Expansion:** Tích chập 1×1 mở rộng số kênh từ C_{in} lên $C_{in} \times t$. Không áp dụng cho block đầu tiên ($t = 1$).
2. **Depthwise Conv:** Tích chập $k \times k$ depthwise trên $C_{in} \times t$ kênh, với stride $s \in \{1, 2\}$.
3. **SE Block (Squeeze-and-Excitation):** Cơ chế attention theo kênh.
4. **Projection (Pointwise Conv):** Tích chập 1×1 chiếu về C_{out} kênh *không* qua activation.
5. **Skip connection:** Cộng residual $\mathbf{X} + \mathbf{Y}$ khi $C_{in} = C_{out}$ và $s = 1$.

Tất cả các lớp trong MBConv (trừ projection cuối) đi kèm Batch Normalization và activation SiLU $f(x) = x \cdot \sigma(x)$.

Squeeze-and-Excitation (SE) Attention. SE block [18] tái hiệu chỉnh các kênh đặc trưng theo cơ chế self-attention. Từ feature map $\mathbf{F} \in \mathbb{R}^{H \times W \times C}$:

1. **Squeeze:** $z_c = \frac{1}{HW} \sum_{i,j} F_{i,j,c}$ – Global Average Pooling nén không gian thành vector $\mathbf{z} \in \mathbb{R}^C$.

2. **Excitation:** $\mathbf{s} = \sigma(W_2 \cdot \delta(W_1 \mathbf{z}))$ – hai lớp FC với bottleneck C/r , δ là ReLU, σ là sigmoid, cho ra vector trọng số $\mathbf{s} \in (0, 1)^C$.
3. **Scale:** $\tilde{F}_{i,j,c} = s_c \cdot F_{i,j,c}$ – nhân từng kênh với trọng số tương ứng, cho phép mô hình chủ động khuếch đại kênh quan trọng và ức chế kênh ít liên quan.

Stochastic Depth. EfficientNet được huấn luyện với kỹ thuật Stochastic Depth [19]: mỗi residual block bị bỏ qua với xác suất $p_l = 1 - \frac{l}{L}(1 - p_{\text{final}})$ tăng tuyến tính theo độ sâu l . Kỹ thuật này hoạt động như một dạng dropout ở cấp độ block, tăng tính regularization và tăng tốc huấn luyện.

Cấu trúc tổng thể EfficientNet-B0. Bảng 1.3 trình bày chi tiết các stage của EfficientNet-B0 với đầu vào $224 \times 224 \times 3$.

Bảng 1.3: Kiến trúc EfficientNet-B0. SE ratio = 0.25; tất cả Conv đi kèm BN + SiLU.

Stage	Operator	Resolution	Channels	Layers	Kernel	Stride	Expansion
1	Conv3×3	112×112	32	1	3×3	2	–
2	MBConv1, SE	112×112	16	1	3×3	1	1
3	MBConv6, SE	56×56	24	2	3×3	2	6
4	MBConv6, SE	28×28	40	2	5×5	2	6
5	MBConv6, SE	14×14	80	3	3×3	2	6
6	MBConv6, SE	14×14	112	3	5×5	1	6
7	MBConv6, SE	7×7	192	4	5×5	2	6
8	MBConv6, SE	7×7	320	1	3×3	1	6
9	Conv1×1 + GAP	1×1	1280	1	1×1	1	–

Sau stage 9, feature vector $\mathbb{R}^{1280}$ đi qua Dropout và lớp Dense. Tổng cộng EfficientNet-B0 có khoảng 5,3 triệu tham số, nhỏ hơn ResNet-50 (~25M) và VGG-16 (~138M) nhưng đạt Top-1 accuracy 77,1% trên ImageNet, vượt trội cả hai.

1.1.8 YOLO-World và Segment Anything Model (SAM)

Trong Hướng 4, nhóm sử dụng kết hợp **YOLO-World** và **SAM** để thực hiện bước phân đoạn vùng thức ăn trước khi phân loại.

YOLO-World [22] được lựa chọn vì khả năng *open-vocabulary detection*, cho phép phát hiện trực tiếp 15 lớp món ăn trong đề tài thông qua text prompt mà không cần huấn luyện lại. Mô hình nhận đầu vào là ảnh và danh sách tên class, sau đó trả về các bounding box tương ứng với độ tin cậy cao. Trong pipeline, kết quả detection của YOLO-World được sử dụng làm vùng quan tâm chính. Trường hợp YOLO-World không phát hiện được đối tượng, nhóm áp dụng chiến lược fallback bằng cách lấy vùng trung tâm chiếm 80% ảnh.

Bounding box thu được từ YOLO-World sau đó được truyền trực tiếp làm **box prompt** cho **SAM-b** [21]. Việc sử dụng box prompt giúp SAM xác định rõ vùng cần phân đoạn, từ đó tạo ra mask nhị phân sát với biên thực của món ăn hơn. Mask thu được từ SAM được sử dụng để crop vùng thức ăn, loại bỏ nền và các thành phần không liên quan trước khi đưa vào mô hình EfficientNet-B0.

Quy trình tổng quát của bước phân đoạn có thể tóm tắt như sau:

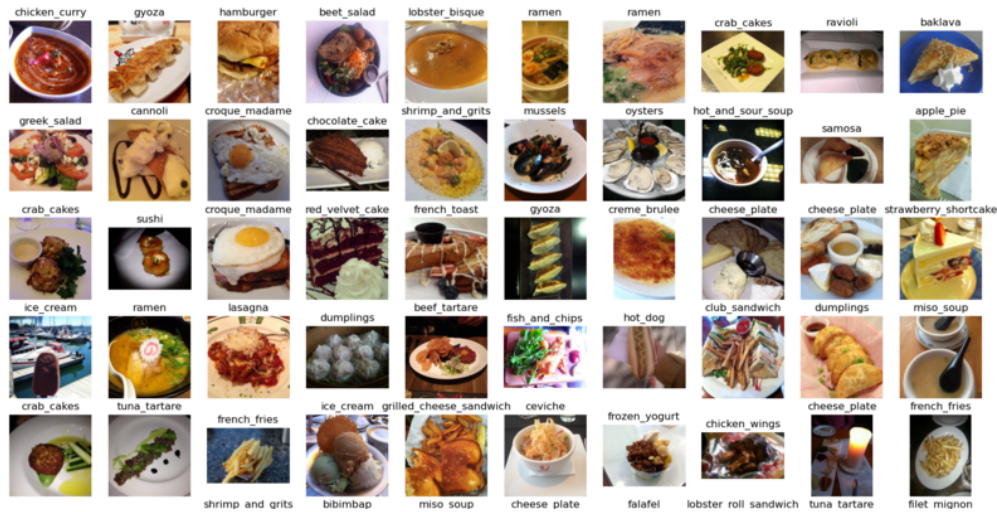
$$\text{bbox}_{\text{YOLO}} \rightarrow \text{SAM}(\text{image}, \text{bbox}_{\text{YOLO}}) \rightarrow \text{mask} \rightarrow \text{Crop vùng thức ăn} \quad (1.10)$$

Kết hợp YOLO-World và SAM cho phép pipeline hoạt động theo hướng *Segmentation-First* mà không cần dữ liệu segmentation ground truth, đồng thời tận dụng được ưu điểm của cả hai mô hình zero-shot.

1.2 Dữ liệu và phương pháp xử lý

1.2.1 Bộ dữ liệu Food-101

Food-101 [2] là bộ dữ liệu chuẩn cho bài toán nhận diện món ăn, bao gồm 101.000 ảnh thuộc 101 loại món ăn (1.000 ảnh/lớp). Mỗi ảnh có độ phân giải tối đa 512×512 pixel ở định dạng JPEG.



Hình 1.1: Một số hình ảnh minh họa từ bộ dữ liệu Food-101

Bộ dữ liệu Food-101 có ưu điểm nổi bật là số lượng ảnh lớn, đa dạng về món ăn và góc chụp, giúp mô hình học được nhiều đặc trưng thị giác khác nhau của thực phẩm. Các ảnh trong tập dữ liệu được thu thập từ thực tế nên chứa nhiều biến thể về ánh sáng, bố cục, kích thước món ăn

và nền ảnh, phù hợp để đánh giá khả năng tổng quát hóa của các mô hình học sâu. Ngoài ra, việc gồm có nhiều lớp món ăn với số lượng ảnh cân bằng giữa các lớp giúp giảm hiện tượng mất cân bằng dữ liệu trong quá trình huấn luyện.

1.2.2 Lựa chọn class thực nghiệm

Tập 15 class – dùng cho tất cả các hướng

Nhóm lựa chọn 15 class dựa trên tiêu chí đa dạng về đặc điểm hình dạng, màu sắc và texture:

- **Shape-dominant:** french_fries, dumplings, ramen, pizza, donuts, hot_dog, pancakes.
- **Color-dominant:** grilled_salmon, ice_cream, chocolate_cake.
- **Texture-dominant:** fried_rice.
- **Amorphous:** omelette, caesar_salad, hamburger, sushi.

Bảng 1.4: Danh sách 15 class chính và đặc điểm phân loại

STT	Tên class	Đặc điểm hình dạng	Nhóm
1	caesar_salad	Vô định hình	Amorphous
2	chocolate_cake	Phẳng, tối	Color-dominant
3	donuts	Hình tròn có lỗ	Shape-dominant
4	dumplings	Cầu, có nếp gấp	Shape-dominant
5	french_fries	Thanh dài song song	Shape-dominant
6	fried_rice	Texture hạt nhỏ	Texture-dominant
7	grilled_salmon	Miếng dẹt, màu cam	Color-dominant
8	hamburger	Nhiều lớp chồng	Amorphous
9	hot_dog	Hình thuôn dài	Shape-dominant
10	ice_cream	Scoop tròn, đa màu	Color-dominant
11	omelette	Blob phẳng đồng nhất	Amorphous
12	pancakes	Tròn phẳng, nhiều lớp	Shape-dominant
13	pizza	Tròn, topping đa dạng	Shape-dominant
14	ramen	Sợi uốn trong tô	Shape-dominant
15	sushi	Đa dạng (roll/nigiri)	Amorphous

Tập 24 class – bổ sung cho phương pháp cổ điển

Bảng 1.5: 9 class bổ sung (24 class)

STT	Tên class	Đặc điểm	Nhóm
1	cheese_plate	Nhiều khối nhỏ đa dạng	Amorphous
2	churros	Thanh dài xoắn, nâu vàng	Shape+Color
3	garlic_bread	Miếng dài, texture cháy	Shape+Color
4	greek_salad	Vô định hình, màu đa dạng	Color-dominant
5	lobster_roll_sandwich	Dạng sandwich dài	Shape-dominant
6	miso_soup	Lỏng, màu vàng-nâu đặc trưng	Color-dominant
7	mussels	Vỏ cứng đen, cụm lại	Shape+Color
8	onion_rings	Vòng tròn đồng tâm	Shape-dominant
9	pho	Nước + sợi (giống ramen)	Shape-dominant

1.2.3 Phân chia tập dữ liệu

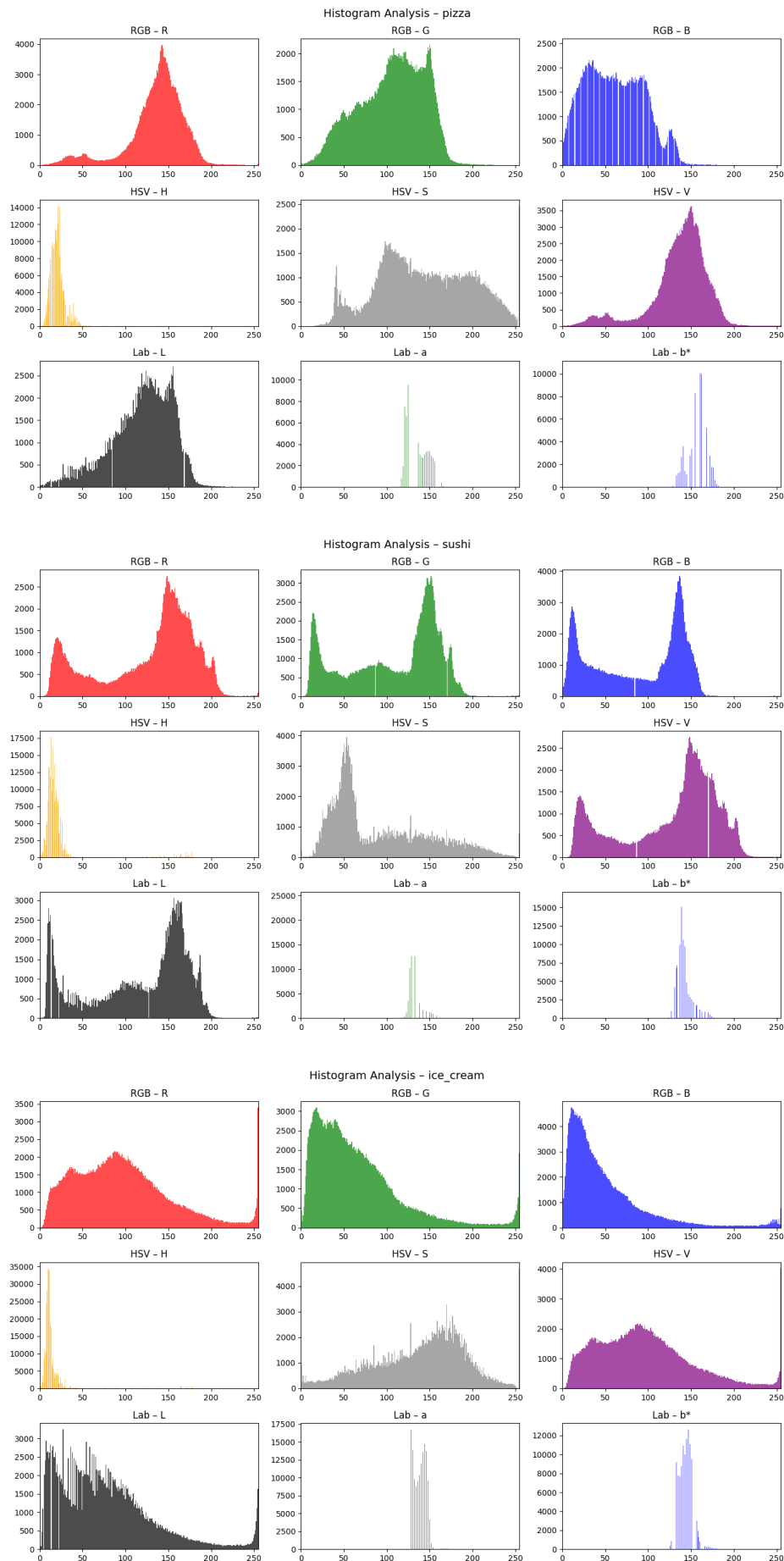
Nhóm thực hiện chia ngẫu nhiên lại toàn bộ dữ liệu bằng `train_test_split` của `scikit-learn` với `random_state=42` và `stratify=y`. Mỗi hướng thực nghiệm được chạy với hai tỷ lệ chia 80/20 và 70/30 để đánh giá tính ổn định.

Bảng 1.6: Số lượng mẫu theo tập dữ liệu và tỷ lệ chia

Tập class	Tổng	Split 80/20		Split 70/30	
		Train	Test	Train	Test
15 class	15.000	12.000	3.000	10.500	4.500
24 class	24.000	19.200	4.800	16.800	7.200

1.2.4 Phân tích phân bố màu theo class

Nhóm trực quan hóa histogram trên các không gian màu cho một số class tiêu biểu. Kết quả cho thấy sự khác biệt đặc trưng về phân bố màu: *pizza* có phân bố màu đa dạng; *grilled_salmon* tập trung rõ rệt trong dải màu cam-hồng đặc trưng; *sushi* biểu hiện sự phân tán màu lớn do tính đa dạng nội lớp.



Hình 1.2: So sánh phân bố màu RGB/HSV/Lab: pizza, sushi và ice_cream

1.2.5 Tiền xử lý ảnh

Lọc ảnh – Bilateral Filter (cho phương pháp cổ điển)

$$I_{\text{filtered}}(p) = \frac{1}{W_p} \sum_{q \in \Omega} I(q) \exp\left(-\frac{\|p - q\|^2}{2\sigma_s^2}\right) \exp\left(-\frac{\|I(p) - I(q)\|^2}{2\sigma_r^2}\right) \quad (1.11)$$

Cài đặt: `cv2.bilateralFilter(d=5, sigmaColor=50, sigmaSpace=50)`.

Tăng cường tương phản – CLAHE

Ảnh được chuyển sang Lab, áp dụng CLAHE trên kênh L , sau đó chuyển lại BGR.

Thứ tự tiền xử lý theo từng hướng

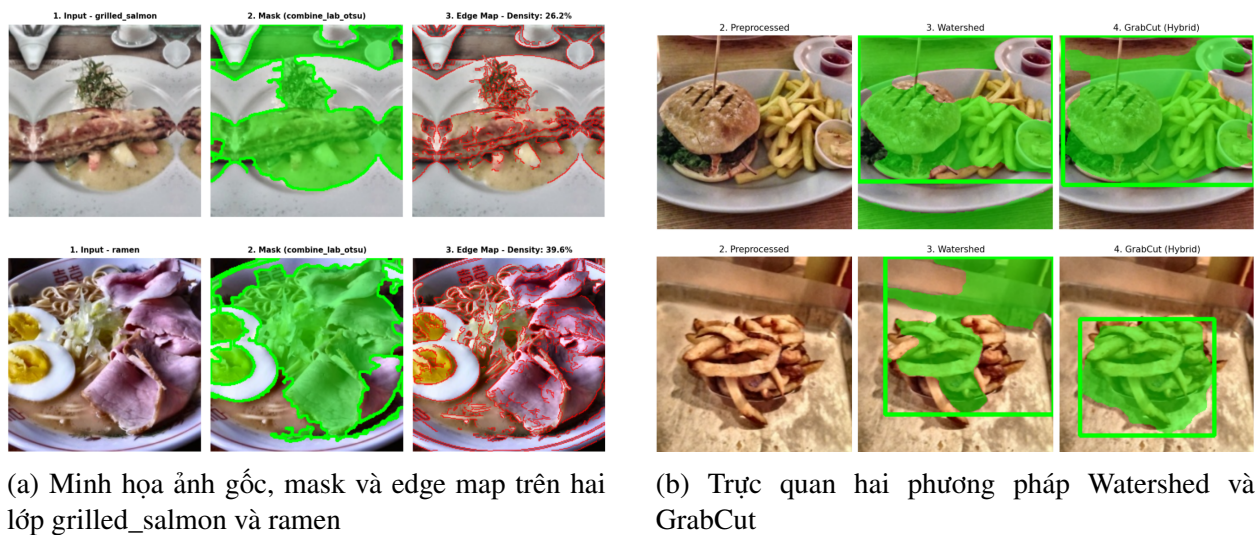
- **Hướng 1:** Bilateral $\rightarrow$ CLAHE $\rightarrow$ Resize (128×128) $\rightarrow$ Grayscale $\rightarrow$ HOG.
- **Hướng 2:** Bilateral $\rightarrow$ CLAHE $\rightarrow$ Resize $\rightarrow \begin{cases} \text{Grayscale} \rightarrow \text{HOG} \\ \text{BGR} \rightarrow \text{HSV} \rightarrow \text{Color Histogram} \end{cases}$
- **Hướng 3, 4:** Resize (224×224 cho CNN)

1.2.6 Phân tích biên và hình thái

Phát hiện biên và phân vùng cổ điển

Nhóm kết hợp hai kỹ thuật: *Edge-based* (ngưỡng tự động trên kênh LAB + Canny) và *Region-based* (ngưỡng Otsu trên kênh S và V của HSV). Kết quả phân tích:

- **Mật độ edge cao:** french_fries, ramen, dumplings, pizza, fried_rice – HOG cho accuracy cao.
- **Mật độ edge thấp:** omelette, grilled_salmon, caesar_salad – HOG accuracy thấp, cần màu sắc hỗ trợ.



Hình 1.3: Trực quan hóa tiền xử lý và phân đoạn ảnh thức ăn

Thao tác hình thái hỗ trợ

Sau khi thu được mask: Closing (kernel 5×5) để nối biên bị đứt; Erosion nhẹ (kernel 3×3) để co vùng mask.

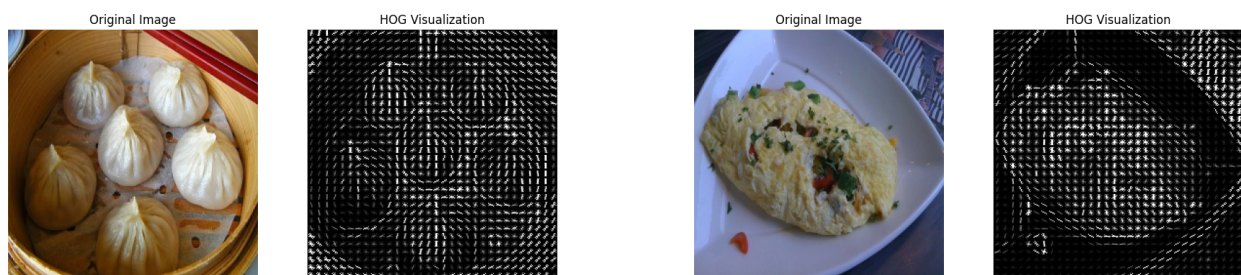
1.2.7 Trích xuất đặc trưng

HOG

Vector HOG 8.100 chiều được tính bằng `skimage.feature.hog` với `block_norm='L2-Hys'`.

Color Histogram HSV

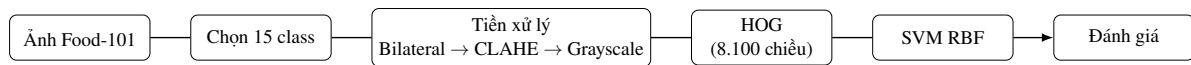
- Kênh H: `cv2.calcHist([hsv], [0], None, [16], [0, 180])`
- Kênh S: `cv2.calcHist([hsv], [1], None, [8], [0, 256])`
- Kênh V: `cv2.calcHist([hsv], [2], None, [8], [0, 256])`

(a) Dumplings – edge nếp gấp rõ $\Rightarrow$ acc. cao(b) Omelette – edge nghèo nàn $\Rightarrow$ acc. thấp

Hình 1.4: So sánh HOG visualization: class có accuracy cao vs. thấp

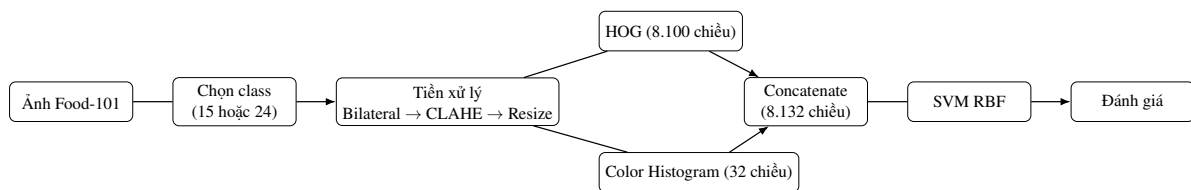
1.2.8 Tổng quan pipeline và các hướng so sánh

Hướng 1: HOG + SVM



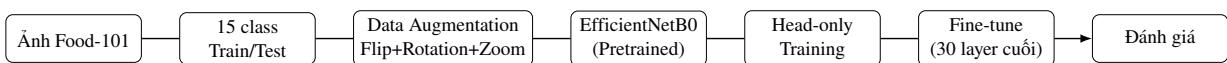
Hình 1.5: Pipeline Hướng 1: HOG + SVM

Hướng 2: HOG + Color Histogram + SVM



Hình 1.6: Pipeline Hướng 2: HOG + Color Histogram + SVM

Hướng 3: Transfer Learning EfficientNet-B0 trên ảnh gốc



Hình 1.7: Pipeline Hướng 3: Transfer Learning EfficientNet-B0 trên ảnh gốc

Hướng 4: Transfer Learning trên SAM Crops (Segmentation-First)



Hình 1.8: Pipeline Hướng 4: Segmentation-First với YOLO-World + SAM

CHƯƠNG 2: THỰC NGHIỆM VÀ ĐÁNH GIÁ – PHƯƠNG PHÁP CỔ ĐIỂN

2.1 Thiết lập thực nghiệm

2.1.1 Cấu hình SVM

Bảng 2.1: Siêu tham số SVM – dùng thống nhất cho cả hai hướng cổ điển

Tham số	Giá trị
Kernel	RBF (rbf)
C	10
γ	scale
Chiến lược đa lớp	One-vs-One (OvO)
Chuẩn hóa đặc trưng	StandardScaler

2.1.2 Các kịch bản thực nghiệm và chỉ số đánh giá

Ba kịch bản thực nghiệm được chạy, mỗi kịch bản trên hai tỷ lệ chia 80/20 và 70/30:

- **Kịch bản 1:** HOG + SVM, 15 class.
- **Kịch bản 2:** HOG + Color Histogram + SVM, 15 class.
- **Kịch bản 3:** HOG + Color Histogram + SVM, 24 class.

Chỉ số đánh giá: **Macro Accuracy** và **Confusion Matrix**.

2.2 Thực nghiệm

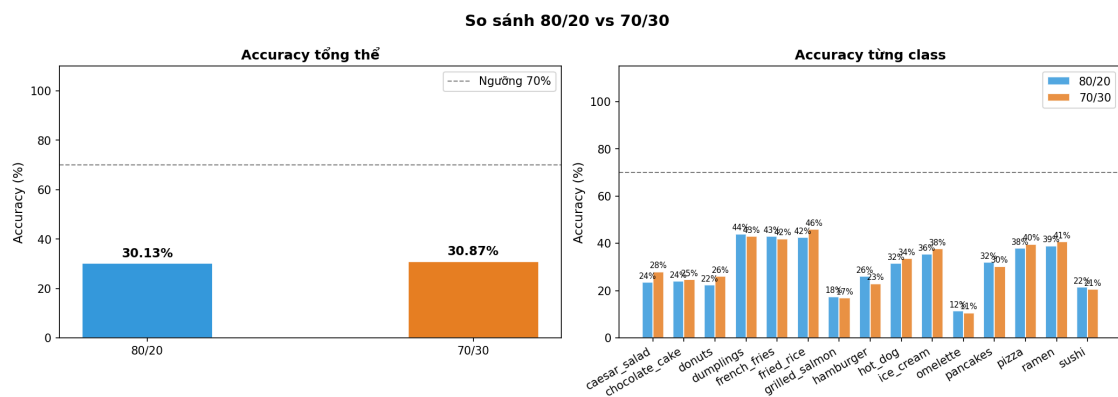
2.2.1 Tổng quan kết quả

Bảng 2.2: Kết quả tổng quan ba hướng thực nghiệm cổ điển

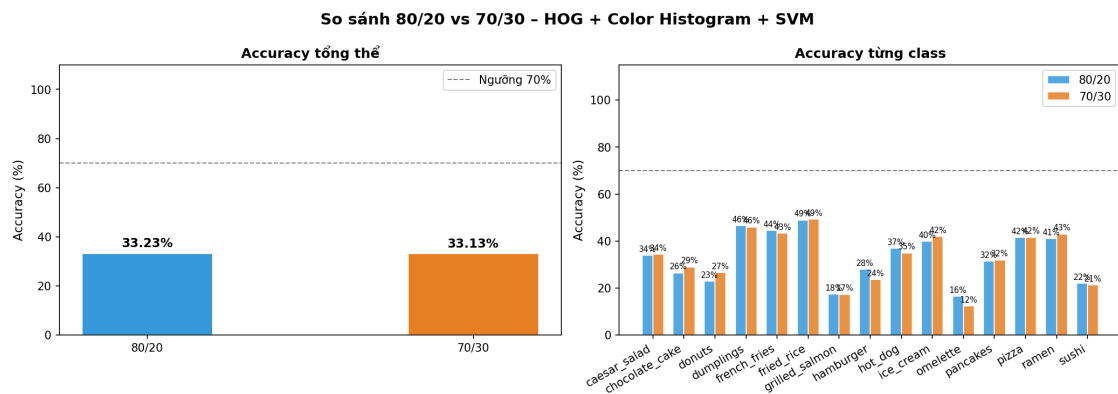
Hướng	Phương pháp	Class	Split	Accuracy (%)	Chiều đặc trưng
1	HOG + SVM	15	80/20	30,13	8.100
1	HOG + SVM	15	70/30	30,87	8.100
2	HOG + CH + SVM	15	80/20	33,23	8.132
2	HOG + CH + SVM	15	70/30	33,13	8.132
3	HOG + CH + SVM	24	80/20	19,20	8.132
3	HOG + CH + SVM	24	70/30	19,30	8.132

2.2.2 So sánh chi tiết: Hướng 1 vs. Hướng 2 trên 15 class

Kết quả theo từng class



(a) Hướng 1: HOG + SVM



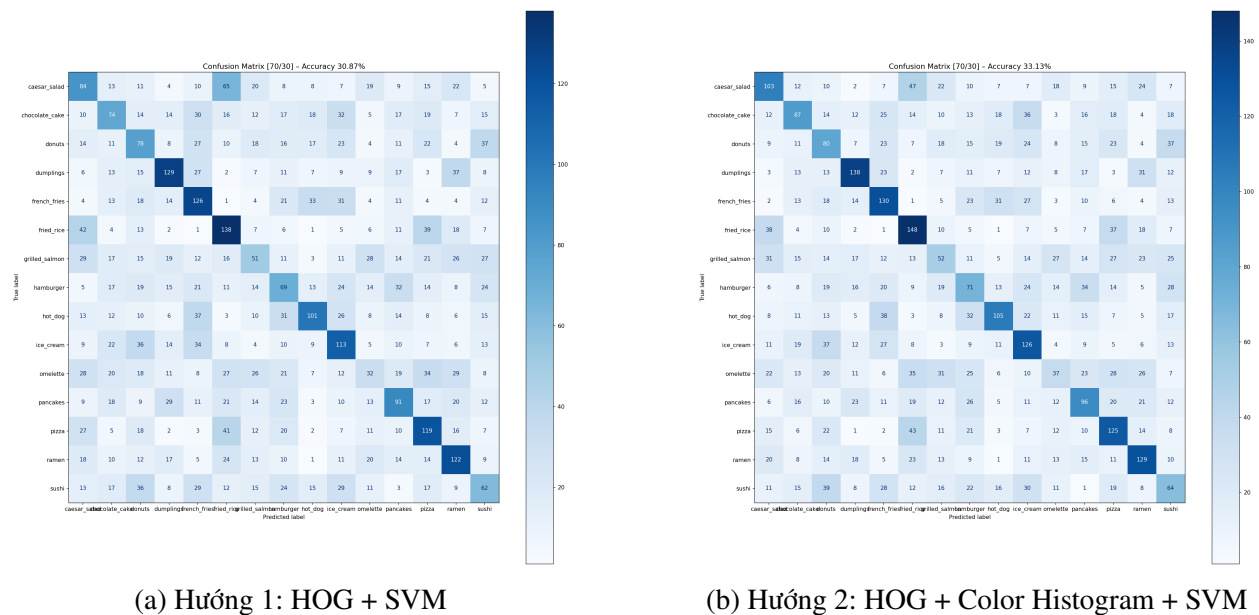
(b) Hướng 2: HOG + Color + SVM

Hình 2.1: So sánh accuracy từng class giữa hai hướng tiếp cận

Bảng 2.3: Accuracy từng class: Hướng 1 vs. Hướng 2 – split 70/30

Class	H1 HOG (%)	H2 HOG+Color (%)	Δ (%)	Giải thích
grilled_salmon	17	17	0	Màu cam/hồng rất đặc trưng
ice_cream	38	42	+4	Màu đa dạng phân biệt tốt
chocolate_cake	25	29	+4	V thấp (tối) rất đặc trưng
french_fries	42	43	+1	Shape mạnh, ít cần màu
dumplings	43	46	+3	Edge nếp gấp rõ ràng
ramen	41	43	+2	Sợi uốn, màu nước bổ trợ
pizza	40	42	+2	Tròn + topping màu sắc
fried_rice	46	49	+3	Texture + màu vàng nâu
omelette	11	12	+1	Blob phẳng, màu vàng nhạt
hamburger	23	24	+1	Đa dạng hình dạng và màu
sushi	21	21	0	Đa dạng nội lớp
caesar_salad	28	34	+6	Vô định hình, màu xanh giúp mạnh
donuts	26	27	+1	Tròn rõ, màu frosting
hot_dog	34	35	+1	Hình dài, màu nâu vàng
pancakes	30	32	+2	Tròn phẳng
Macro avg	31.0	33.1	+2.1	

So sánh Confusion Matrix



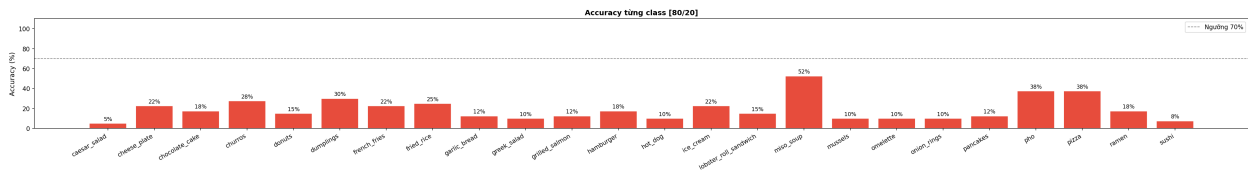
Hình 2.2: So sánh Confusion Matrix: Hướng 1 vs. Hướng 2 (split 70/30)

Các cặp nhầm lẫn đáng chú ý:

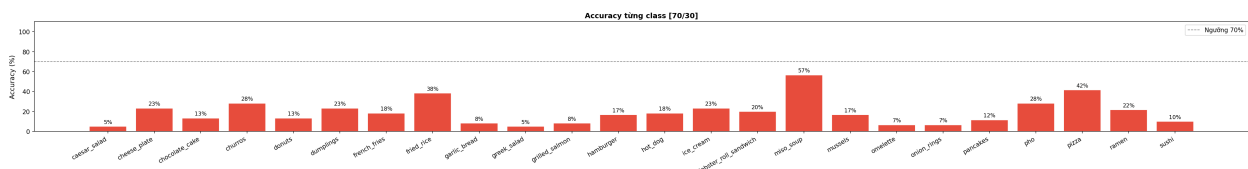
- **omelette** $\leftrightarrow$ **pancakes**: Hướng 1 nhầm nhiều (cả hai blob phẳng vàng khi grayscale). Hướng 2 giảm nhầm lẫn nhờ phân bố V khác nhau.
- **hamburger** $\leftrightarrow$ **hot_dog**: Cả hai có bánh mì và nhân thịt – màu sắc không giúp nhiều vì gam màu vàng–nâu giống nhau.

- **grilled_salmon** ↔ **các class khác**: Hướng 2 phân tách rõ nhờ kênh H tập trung hoàn toàn khác.
- **sushi** ↔ **dumplings/ramen**: Cả ba là món châu Á – sự đa dạng ngữ nghĩa trong class sushi khiến cả hai phương pháp đều khó phân biệt.

2.2.3 Kết quả: HOG + Color Histogram + SVM trên 24 class



Hình 2.3: Accuracy từng class – 24 class, Split 80/20



Hình 2.4: Accuracy từng class – 24 class, Split 70/30

Bảng 2.4: Accuracy từng class – HOG + Color Histogram + SVM, 24 class

Class	Split 80/20 (%)	Split 70/30 (%)	Nhóm
caesar_salad	5	5	Amorphous
cheese_plate	22	23	Amorphous
chocolate_cake	18	13	Color-dominant
churros	28	28	Shape+Color
donuts	15	13	Shape-dominant
dumplings	30	23	Shape-dominant
french_fries	22	18	Shape-dominant
fried_rice	25	38	Texture-dominant
garlic_bread	12	8	Shape+Color
greek_salad	10	5	Color-dominant
grilled_salmon	12	8	Color-dominant
hamburger	18	17	Amorphous
hot_dog	10	18	Shape-dominant
ice_cream	22	23	Color-dominant
lobster_roll_sandwich	15	20	Shape-dominant
miso_soup	52	57	Color-dominant
mussels	10	17	Shape+Color
omelette	10	7	Amorphous
onion_rings	10	7	Shape-dominant
pancakes	12	12	Shape-dominant
pho	38	28	Shape-dominant
pizza	38	42	Shape-dominant
ramen	18	22	Shape-dominant
sushi	8	10	Amorphous
Macro avg	19,2	19,3	

2.3 Phân tích và thảo luận

2.3.1 Phân tích nguyên nhân cải thiện khi bổ sung Color Histogram

Mức cải thiện của Hướng 2 so với Hướng 1 là không đồng đều, có thể phân tích theo bốn cơ chế:

Cơ chế 1 – Bổ sung màu cho class phụ thuộc màu: *caesar_salad* (+6%), *ice_cream* và *chocolate_cake* (+4%) được hưởng lợi nhiều nhất do có màu sắc đặc trưng và ổn định.

Cơ chế 2 – Kết hợp shape và color: *dumplings* (+3%), *fried_rice* (+3%), *ramen*, *pizza*, *pancakes* (+2%) được cải thiện nhờ sự bổ trợ giữa HOG và histogram màu.

Cơ chế 3 – Ít cải thiện khi shape đã đủ mạnh: *french_fries*, *donuts*, *hot_dog* chỉ tăng nhẹ (+1%).

Cơ chế 4 – Không cải thiện khi đặc trưng không ổn định: *grilled_salmon* và *sushi* (0%) do màu sắc không ổn định hoặc nội lớp quá đa dạng.

2.3.2 Phân tích kết quả 24 class

miso_soup (52%/57%) là class nổi bật nhất – màu vàng–nâu đặc trưng rất nhất quán, phân bố Hue tập trung hẹp tại $H \approx 15\text{--}30$, là ví dụ điển hình Color Histogram phát huy tối đa. **Cặp ramen/pho** vẫn bị nhầm lẫn dù màu nước khác nhau vì cấu trúc sợi HOG quá giống nhau.

2.3.3 Thảo luận về giới hạn của phương pháp cổ điển

1. **Mất thông tin màu sắc (HOG):** Không thể khắc phục bằng cách tối ưu HOG hay SVM.
2. **Bỏ qua thông tin không gian (Color Histogram):** Histogram toàn cục không biết vị trí màu trong ảnh.
3. **Nhiều màu nền:** Histogram tính trên toàn ảnh bao gồm cả đĩa, bàn, nền.
4. **Khoảng cách với Deep Learning:** ResNet-50 với transfer learning đạt $\sim 85\%$ trên toàn bộ 101 class Food-101 – cao hơn gần gấp đôi so với HOG + Color Histogram + SVM trên tập con của nhóm.

CHƯƠNG 3: HỌC SÂU, ƯỚC LƯỢNG KHẤU PHẦN VÀ ỨNG DỤNG

3.1 Thiết lập thực nghiệm học sâu

3.1.1 Môi trường và cấu hình phần cứng

Bảng 3.1: Cấu hình phần cứng và môi trường thực nghiệm học sâu

Thành phần	Cấu hình
Nền tảng	Kaggle Notebooks
GPU	2× Tesla T4 (15 GB VRAM mỗi card)
Framework	TensorFlow 2.x / Keras
Mixed precision	<code>mixed_float16</code>

3.1.2 Cấu hình huấn luyện EfficientNet-B0

Bảng 3.2: Cấu hình huấn luyện EfficientNet-B0

Tham số	Giai đoạn 1 (Head-only)	Giai đoạn 2 (Fine-tune)
Optimizer	Adam	Adam
Learning rate	10^{-3}	10^{-5}
Batch size	32	32
Số epoch tối đa	5	20
Backbone layers	Frozen (all)	Unfreeze 30 layer cuối
Early stopping	patience=3	patience=5
ReduceLROnPlateau	factor=0.2, p=2	factor=0.2, p=2
Loss function	Sparse Categorical Crossentropy	idem
Input shape	(224, 224, 3)	idem

Data Augmentation:

- `RandomFlip("horizontal")`: lật ngang ngẫu nhiên
- `RandomRotation(0.1)`: xoay $\pm 36^\circ$
- `RandomZoom(0.1)`: thu phóng $\pm 10\%$
- `RandomContrast(0.1)`: thay đổi tương phản $\pm 10\%$

3.1.3 Thực nghiệm Transfer Learning

Hướng 3: EfficientNet-B0 trên ảnh gốc

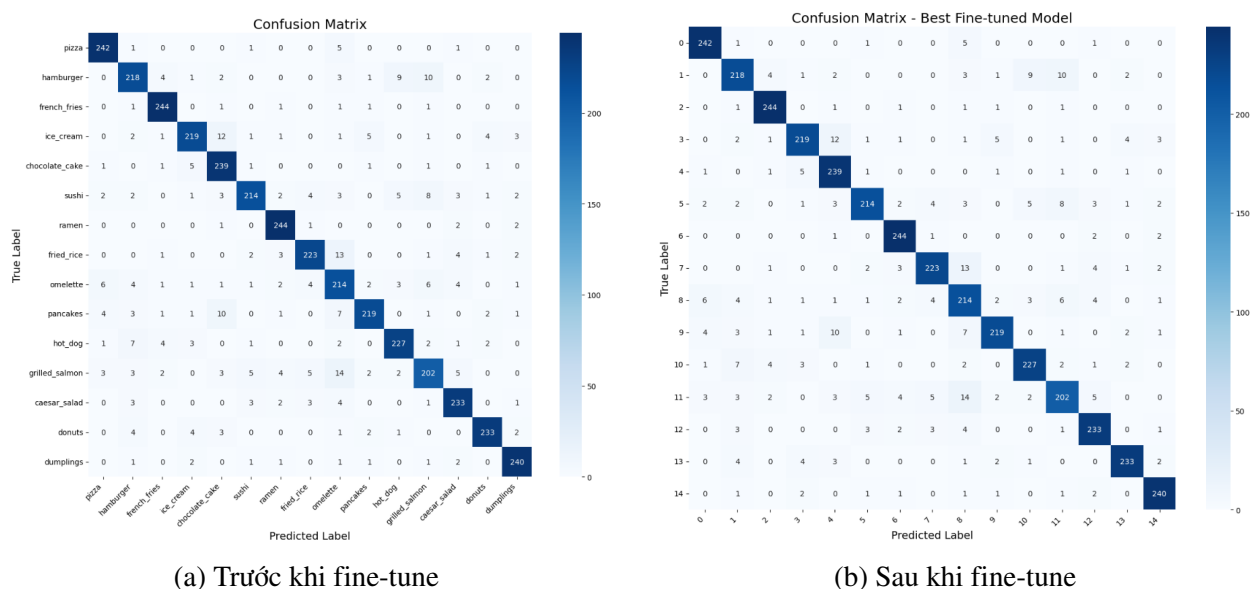
Pipeline và cấu hình.

- **Augmentation:** RandomFlip (horizontal), RandomRotation($\pm 10\%$), RandomZoom($\pm 10\%$), RandomContrast($\pm 10\%$) – đặt trong model, chỉ kích hoạt khi training.
- **Kiến trúc:** EfficientNet-B0 pretrained ImageNet + GlobalAveragePooling2D + Dropout(0.2) + Dense(15, softmax).
- **Giai đoạn 1:** Đóng băng backbone, train classification head với Adam $\eta = 10^{-3}$, sparse_categorical_crossentropy, tối đa 5 epoch, EarlyStopping patience 3.
- **Giai đoạn 2:** Mở băng 30 layer cuối, fine-tune với Adam $\eta = 10^{-5}$, tối đa 20 epoch, EarlyStopping patience 5.

Kết quả phân loại. Mô hình CNN sau khi fine-tune đạt độ chính xác tổng thể (Accuracy) là **90.96%** trên tập kiểm thử gồm 3750 ảnh thuộc 15 lớp món ăn. Các lớp đạt kết quả tốt nhất bao gồm *french_fries*, *ramen*, *pizza* và *dumplings* với F1-score đều lớn hơn 95%.

Một số lớp như *omelette* và *grilled_salmon* có kết quả thấp hơn do sự tương đồng về màu sắc, kết cấu và hình dạng giữa các món ăn trong tập dữ liệu.

Kết quả đánh giá. Hình 3.1a trình bày confusion matrix của mô hình trước khi fine-tune, trong khi Hình 3.1b thể hiện confusion matrix của mô hình sau khi fine-tune. Có thể quan sát rằng sau quá trình fine-tune, số lượng mẫu được phân loại đúng trên đường chéo chính tăng lên đáng kể và hiện tượng nhầm lẫn giữa các lớp được giảm bớt.



Hình 3.1: Confusion Matrix trước và sau khi fine-tune

Bảng 3.3 trình bày chi tiết các chỉ số Precision, Recall và F1-score của mô hình CNN sau khi fine-tune.

Bảng 3.3: Kết quả classification report của mô hình CNN sau fine-tune

Lớp	Precision	Recall	F1-score
pizza	0.934	0.968	0.951
hamburger	0.876	0.872	0.874
french_fries	0.942	0.976	0.959
ice_cream	0.924	0.876	0.899
chocolate_cake	0.869	0.956	0.910
sushi	0.930	0.856	0.892
ramen	0.935	0.976	0.955
fried_rice	0.929	0.892	0.910
omelette	0.796	0.856	0.825
pancakes	0.936	0.876	0.905
hot_dog	0.919	0.908	0.913
grilled_salmon	0.860	0.808	0.833
caesar_salad	0.914	0.932	0.923
donuts	0.947	0.932	0.940
dumplings	0.945	0.960	0.952
Macro Avg	0.910	0.910	0.909
Weighted Avg	0.910	0.910	0.909

Phân tích. Kết quả thực nghiệm cho thấy quá trình fine-tune giúp cải thiện đáng kể khả năng phân loại của mô hình CNN trên tập dữ liệu món ăn. Các lớp có đặc trưng hình học và màu sắc rõ ràng như *pizza*, *ramen*, *french_fries* và *dumplings* đạt F1-score trên 95%, cho thấy mô hình học tốt các đặc trưng thị giác đặc trưng của từng món ăn.

Ngược lại, các lớp như *omelette* và *grilled_salmon* có kết quả thấp hơn do sự đa dạng lớn

về cách trình bày món ăn cũng như sự tương đồng về màu sắc và kết cấu với các lớp khác trong tập dữ liệu.

Ngoài ra, confusion matrix sau fine-tune cho thấy hiện tượng nhầm lẫn giữa các lớp đã giảm đáng kể so với trước fine-tune, đặc biệt ở các nhóm món ăn có đặc trưng gần giống nhau. Nhìn chung, mô hình đạt macro F1-score khoảng **90.9%**, cho thấy khả năng tổng quát hóa tốt trên tập kiểm thử.

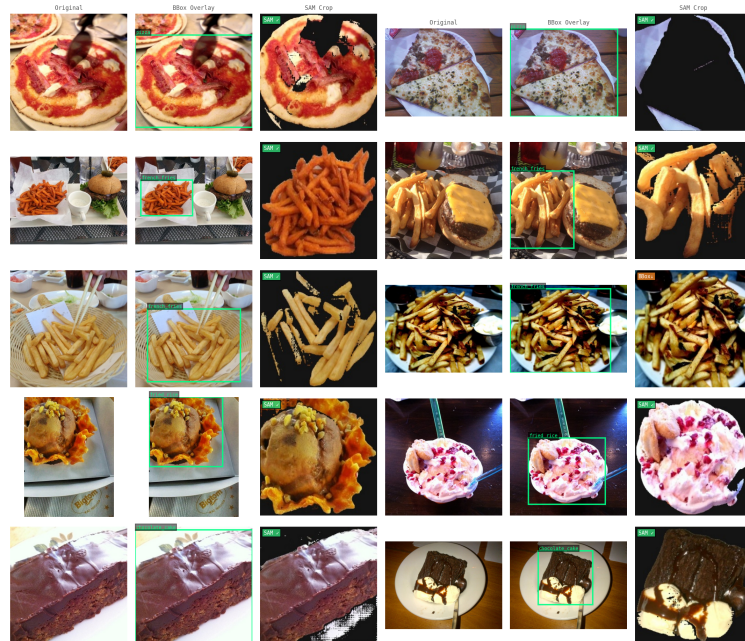
3.1.4 Thực nghiệm EfficientNet-B0 trên SAM Crops (Segmentation-First)

Kết quả phát hiện và phân đoạn Bảng 3.4 tổng hợp hiệu suất trên 15.000 ảnh (1.000 ảnh/lớp).

Bảng 3.4: Kết quả định lượng YOLO-World và SAM

Lớp	Phát hiện (%)	Tin cậy TB	SAM thành công (%)
pizza	98.0	0.902	89.2
fried_rice	97.6	0.905	91.9
french_fries	96.5	0.776	80.1
hot_dog	96.4	0.834	92.7
hamburger	94.9	0.744	91.0
ramen	94.8	0.748	87.5
omelette	84.4	0.597	79.9
pancakes	84.2	0.523	79.9
chocolate_cake	83.3	0.564	81.5
caesar_salad	81.7	0.624	71.7
donuts	78.8	0.506	78.1
grilled_salmon	69.9	0.563	67.6
ice_cream	60.5	0.485	58.6
sushi	47.9	0.525	46.6
dumplings	44.8	0.506	43.5
TB	80.9	0.654	76.0

Các lớp có hình dạng đặc trưng rõ ràng (pizza, fried_rice, hot_dog) đạt tỷ lệ phát hiện và SAM thành công cao (>87%). Ngược lại, sushi và dumplings có tỷ lệ thấp nhất (<47%), phần lớn phải dùng center-crop fallback do YOLO không nhận diện được. Tuy nhiên kết quả của mask có thể lỗi do **SAM chọn sai vùng foreground** — xảy ra khi bbox từ YOLO bao gồm cả đĩa hoặc nền, khiến SAM ưu tiên segment vùng có ranh giới tương phản rõ hơn thay vì toàn bộ món ăn; biểu hiện điển hình là các crop *pizza* xuất hiện vùng đen lớn ngay giữa ảnh, thậm chí gần như toàn bộ crop bị tô đen.

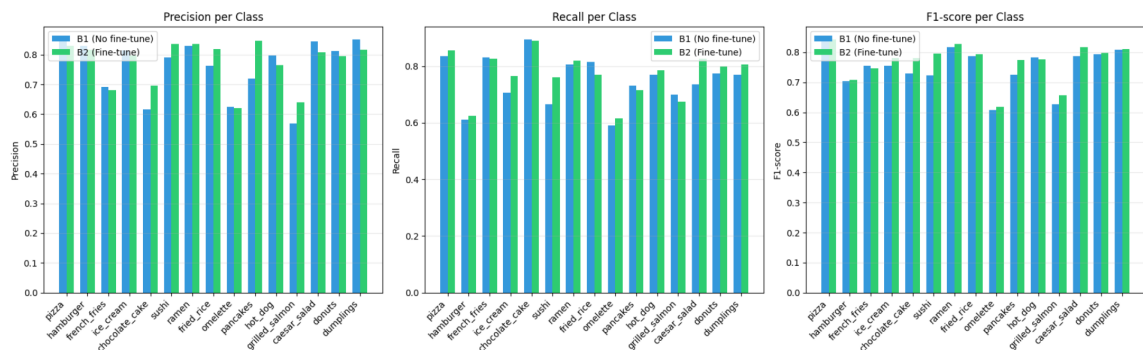


Hình 3.2: Minh họa kết quả pipeline: Original - YOLO BBox Overlay - SAM Crop

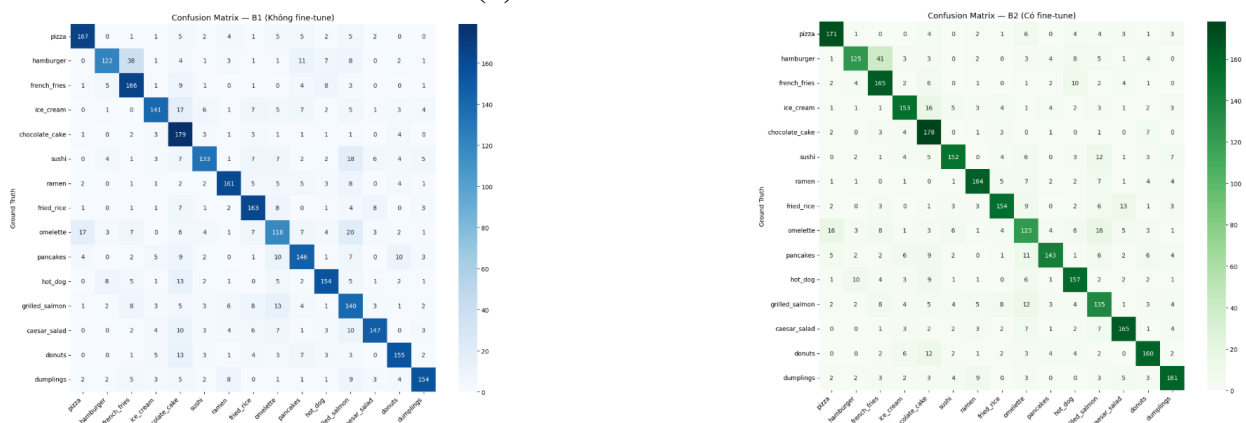
Kết quả phân loại

(a) So sánh các chỉ số Precision, Recall, F1-score

So sánh các chỉ số giữa B1 và B2



(b) Confusion Matrix



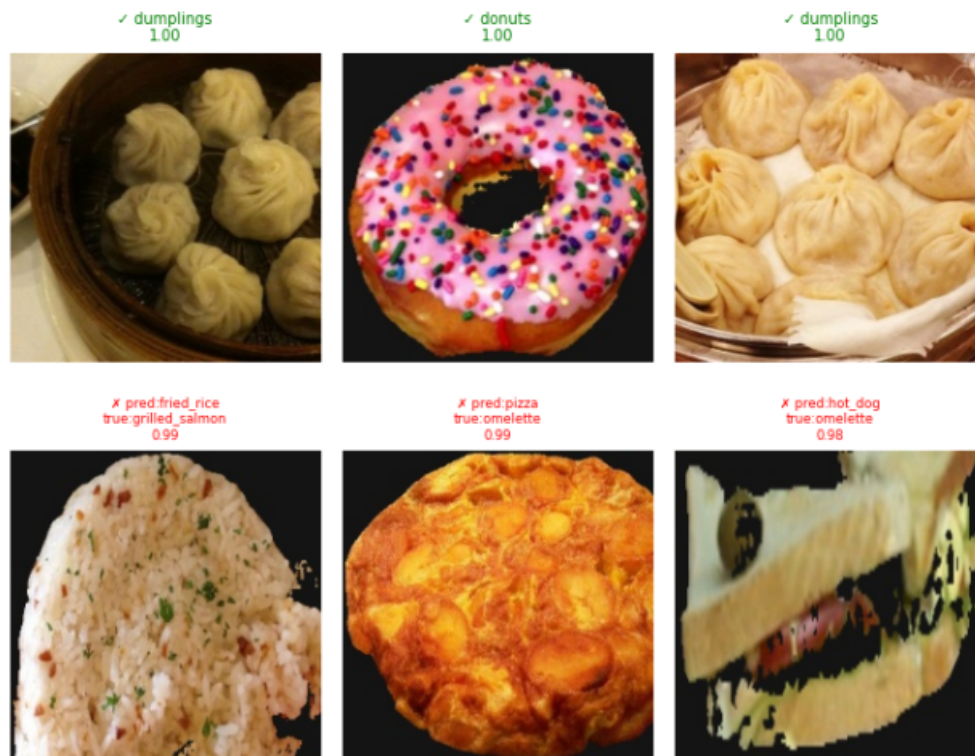
(a) B1 (Không fine-tune)

(b) B2 (Có fine-tune)

Hình 3.3: Tổng hợp kết quả giữa B1 và B2

Phân tích định lượng. Mô hình fine-tune đạt độ chính xác **76,87%**, cao hơn B1 khoảng 2%; loss giảm từ 0.8766 xuống 0.7831. Các lớp *omelette*, *grilled_salmon* và *hamburger* vẫn duy trì hiệu suất thấp ở cả hai mô hình; omelette là lớp yếu nhất với $F1 \approx 0.618$.

Phân tích định tính.



Hình 3.4: Kết quả phân loại

Các trường hợp phân loại đúng chủ yếu thuộc về lớp có vùng thức ăn được SAM segment rõ và chiếm tỷ lệ lớn trong khung hình (*pizza*, *donuts*, *dumplings*, *chocolate_cake*). Phân loại sai tập trung ở các nguyên nhân: phân vùng nhầm món trong ảnh, ảnh sau phân vùng bị thiếu ngữ cảnh nên *omelette* bị nhầm với các lớp có hình dạng và màu sắc tương đồng. Những lỗi này phản ánh hạn chế cốt lõi của pipeline: EfficientNet-B0 học tốt nhưng chất lượng đặc trưng đầu vào bị ràng buộc bởi độ chính xác của YOLO-World và cơ chế chọn mask của SAM. Cải thiện giai đoạn segmentation là điều kiện cần để nâng hiệu suất tổng thể.

3.2 So sánh tổng hợp bốn phương pháp

Bảng 3.5: Bảng so sánh tổng hợp bốn phương pháp (15 class)

Phương pháp	Top-1 Acc (%)	Macro-F1	Đặc trưng
HOG + SVM (H1)	30–31	–	Thủ công (8.100)
HOG + Color + SVM (H2)	33	–	Thủ công (8.132)
EfficientNet-B0 (H3)	90.96	0.909	Học tự động
EfficientNet + SAM (H4)	76.87	0.7685	Học tự động + Segmentation

Bốn hướng tiếp cận cho thấy sự khác biệt giữa phương pháp cổ điển và phương pháp học sâu. Trong khi các phương pháp cổ điển sử dụng đặc trưng thủ công chỉ khai thác thông tin ở mức thấp (cạnh, gradient và phân bố màu), thì mạng nơ-ron tích chập có khả năng học được các đặc trưng phân cấp, từ mức thấp đến mức ngữ nghĩa cao. Điều này giúp mô hình học sâu thích nghi tốt hơn với sự đa dạng trong dữ liệu. Do đó, khoảng cách về hiệu năng giữa hai nhóm phương pháp phản ánh đúng bản chất của từng hướng tiếp cận.

Đối với nhóm phương pháp cổ điển, việc kết hợp đặc trưng hình dạng (HOG) với thông tin màu sắc (Color Histogram) mang lại sự cải thiện nhất định so với việc chỉ sử dụng đặc trưng hình dạng đơn thuần. Tuy nhiên, mức độ cải thiện này không đồng đều giữa các lớp và gặp hạn chế rõ rệt khi số lượng lớp tăng lên. Những hạn chế này cho thấy phương pháp cổ điển tuy có ưu điểm về tính giải thích được và chi phí tính toán thấp, nhưng khó đáp ứng yêu cầu của bài toán phân loại món ăn thực tế với độ phức tạp cao.

Đối với pipeline Segmentation-First, hiệu quả của hướng này hiện vẫn bị ràng buộc chặt chẽ bởi chất lượng của giai đoạn phân đoạn. Khi mô hình phân đoạn chưa đủ chính xác, các lỗi ở bước này sẽ lan truyền sang giai đoạn phân loại, đồng thời việc cắt vùng cũng có thể làm mất đi một phần ngữ cảnh quan trọng của món ăn. EfficientNet-B0 với Transfer Learning đạt kết quả vượt trội. Mô hình không chỉ nắm bắt được hình dạng mà còn học được texture, màu sắc và ngữ cảnh cục bộ một cách hiệu quả. Các lớp có cấu trúc rõ ràng (pizza, ramen, french_fries, dumplings) đạt F1-score > 0.95, trong khi các lớp có biến thiên lớn (omelette, grilled_salmon) vẫn là điểm yếu chung của mọi phương pháp.

3.3 Ước lượng khẩu phần ăn

3.3.1 Phạm vi và giới hạn bài toán

Giới hạn của Food-101 đối với bài toán ước lượng khẩu phần

Food-101 được thiết kế cho bài toán nhận diện, không phải ước lượng khẩu phần. Ba vấn đề cốt lõi không thể giải quyết bằng thuật toán:

- **Không có nhãn khẩu phần:** Dataset chỉ có nhãn class, không có thông tin gram, calorie hay kích thước.
- **Khoảng cách camera không kiểm soát:** Cùng một đĩa pizza có thể chiếm 20% hoặc 90% khung hình tùy người chụp.
- **Góc chụp đa dạng:** Mỗi góc cho diện tích pixel khác nhau dù món ăn thật không thay đổi.

Định nghĩa bài toán đơn giản hóa

Do các giới hạn trên, nhóm định nghĩa lại bài toán ở mức thô: **ước lượng sự hiện diện của thức ăn trong ảnh** và phân loại thành ba mức:

- **Small:** Thức ăn chiếm $< 1/3$ khung hình, hoặc là một phần nhỏ của món.
- **Medium:** Thức ăn chiếm $1/3 - 2/3$ khung hình.
- **Large:** Thức ăn chiếm $> 2/3$ khung hình, hoặc cả đĩa/tô đầy.

Giả thiết rõ ràng: Góc chụp nhìn từ trên xuống, khoảng cách camera tương đối nhất quán. Nhóm thành thật công nhận đây là ước lượng thô và không có giá trị chính xác với bộ dữ liệu này.

3.3.2 Pipeline ước lượng khẩu phần

Pipeline kế thừa mask từ SAM (bước 3 của Hướng 4) và tính 5 đặc trưng hình học:

Trích xuất đặc trưng hình học (Geometry Ratios)

Bảng 3.6: Năm đặc trưng hình học dùng để ước lượng khẩu phần

Ký hiệu	Tên	Trọng số	Ý nghĩa
R1	Area ratio	0.50	$\frac{\text{số pixel foreground}}{\text{tổng số pixel ảnh}}$ – thức ăn chiếm bao nhiêu % khung hình
R2	BBox fill ratio	0.25	$\frac{\text{diện tích foreground}}{\text{diện tích bounding box}}$ – thức ăn đặc hay thưa trong vùng của nó
R3	Compactness	0.05	$\frac{4\pi A}{P^2}$ (1 = tròn, 0 = loảng ngoảng) – hình dạng tổng thể
R4	Saturation mean	0.10	Mean kênh S (HSV) trong foreground – màu sắc phong phú trông “nhiều” hơn
R5	Edge density	0.10	$\frac{\text{số pixel cạnh Canny trong FG}}{\text{fg_pixels}}$ – texture phong phú (nhiều topping)

Tính Weighted Score và phân loại S/M/L

$$s = \sum_{k \in \{R1, \dots, R5\}} w_k \cdot r_k, \quad s \in [0, 1] \quad (3.1)$$

Phân loại theo ngưỡng, với hiệu chỉnh per-class offset δ_c :

$$\text{Portion} = \begin{cases} \text{Small} & \text{nếu } s < T_{\text{small}} + \delta_c \\ \text{Large} & \text{nếu } s > T_{\text{large}} + \delta_c \\ \text{Medium} & \text{otherwise} \end{cases} \quad (3.2)$$

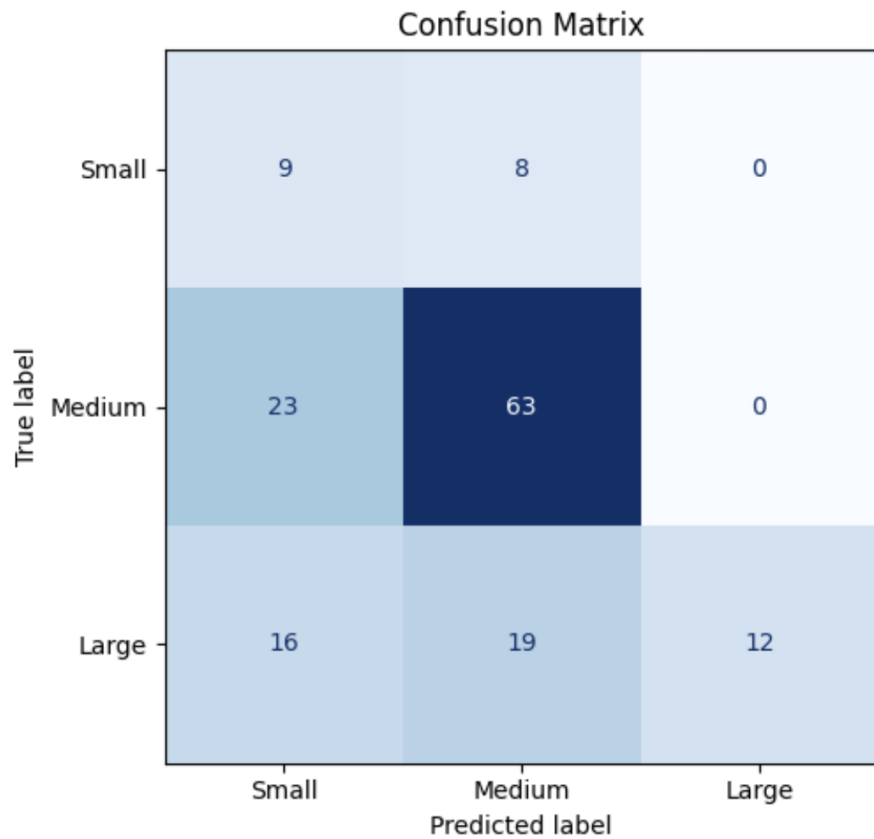
với $T_{\text{small}} = 0.28$, $T_{\text{large}} = 0.55$.

Per-class offset δ_c điều chỉnh ngưỡng theo đặc thù từng class (ví dụ: ramen thường chụp gần, chiếm nhiều frame hơn thực tế khẩu phần $\Rightarrow \delta_{\text{ramen}} = +0.08$).

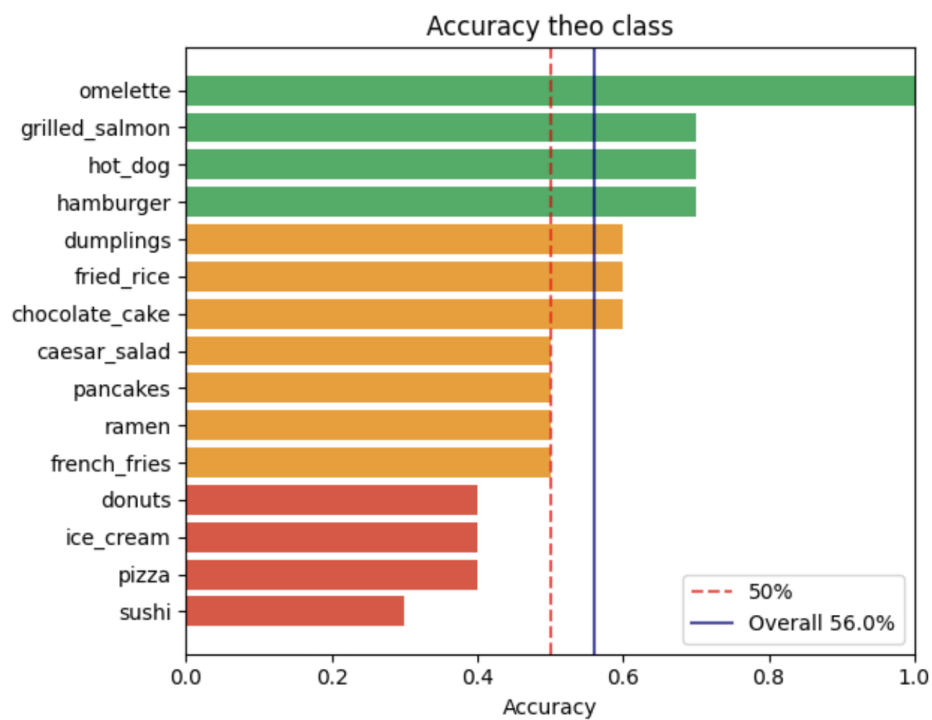
Gán nhãn ground truth thủ công

Do Food-101 không có nhãn khẩu phần, nhóm gán nhãn thủ công cho 10 ảnh/class $\times$ 15 class = 150 ảnh, dựa trên tiêu chí S/M/L định nghĩa ở trên. Nhãn này được dùng để đánh giá định lượng hệ thống.

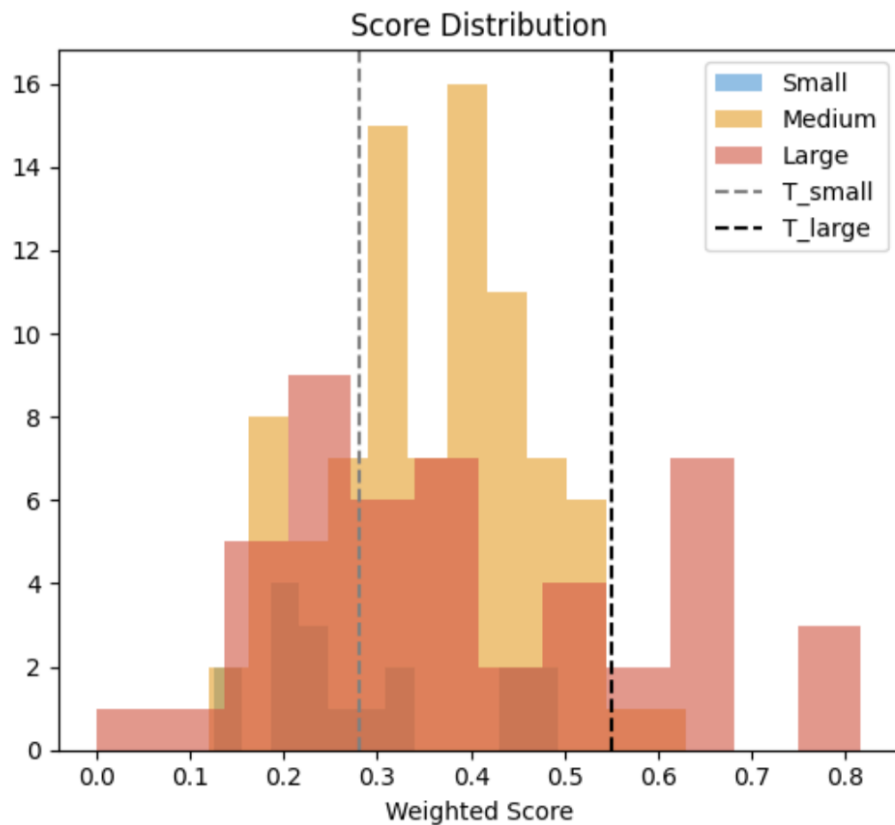
Kết quả đánh giá hệ thống



Hình 3.5: Confusion Matrix của hệ thống phân loại khẩu phần



Hình 3.6: Độ chính xác theo từng lớp món ăn (15 classes)



Hình 3.7: Phân bố điểm dự đoán khẩu phần

Nhận xét, đánh giá

1. **Chủ yếu tập trung ở mức Medium** vì chưa thực sự phân rõ thức ăn ở 3 lớp khác nhau theo tiêu chí thế nào, việc thức ăn quá nhỏ hoặc quá lớn sẽ được đánh nhãn là Small hoặc Large, tuy nhiên, khi chụp gần thức ăn nhỏ thì mô hình dựa vào 5 tiêu chí lại đoán là Large, ngược lại, nếu thức ăn nhiều nhưng chụp ra xa, khiến food chiếm ít pixel trong ảnh thì lại dự đoán là Small.
2. **Dán nhãn thủ công** Do không có nhãn thật, nhóm tự dán nhãn 150 ảnh thuộc 15 classes khác nhau, tuy nhiên do các vấn đề về góc chụp, cách chụp nên nhóm có thể còn đánh nhãn sai, không thật sự là 100% với bộ dữ liệu này, số lượng ảnh được gán nhãn ground truth thủ công còn khá nhỏ (150 ảnh), nên chưa phản ánh đầy đủ toàn bộ sự đa dạng của dữ liệu thực tế. Việc đánh giá vì vậy vẫn còn mang tính tham khảo và có thể chưa bao quát hết các trường hợp khó.
3. **Khó khăn rất lớn khiến kết quả không hoàn toàn là đúng** Trước hết, bộ dữ liệu Food-101 được thu thập từ nhiều nguồn khác nhau nên ảnh có sự đa dạng rất lớn về góc chụp, khoảng cách chụp và điều kiện ánh sáng. Với cùng một món ăn, ảnh chụp cận cảnh thường làm món ăn chiếm phần lớn diện tích khung hình, trong khi ảnh chụp từ xa khiến món ăn nhỏ hơn đáng

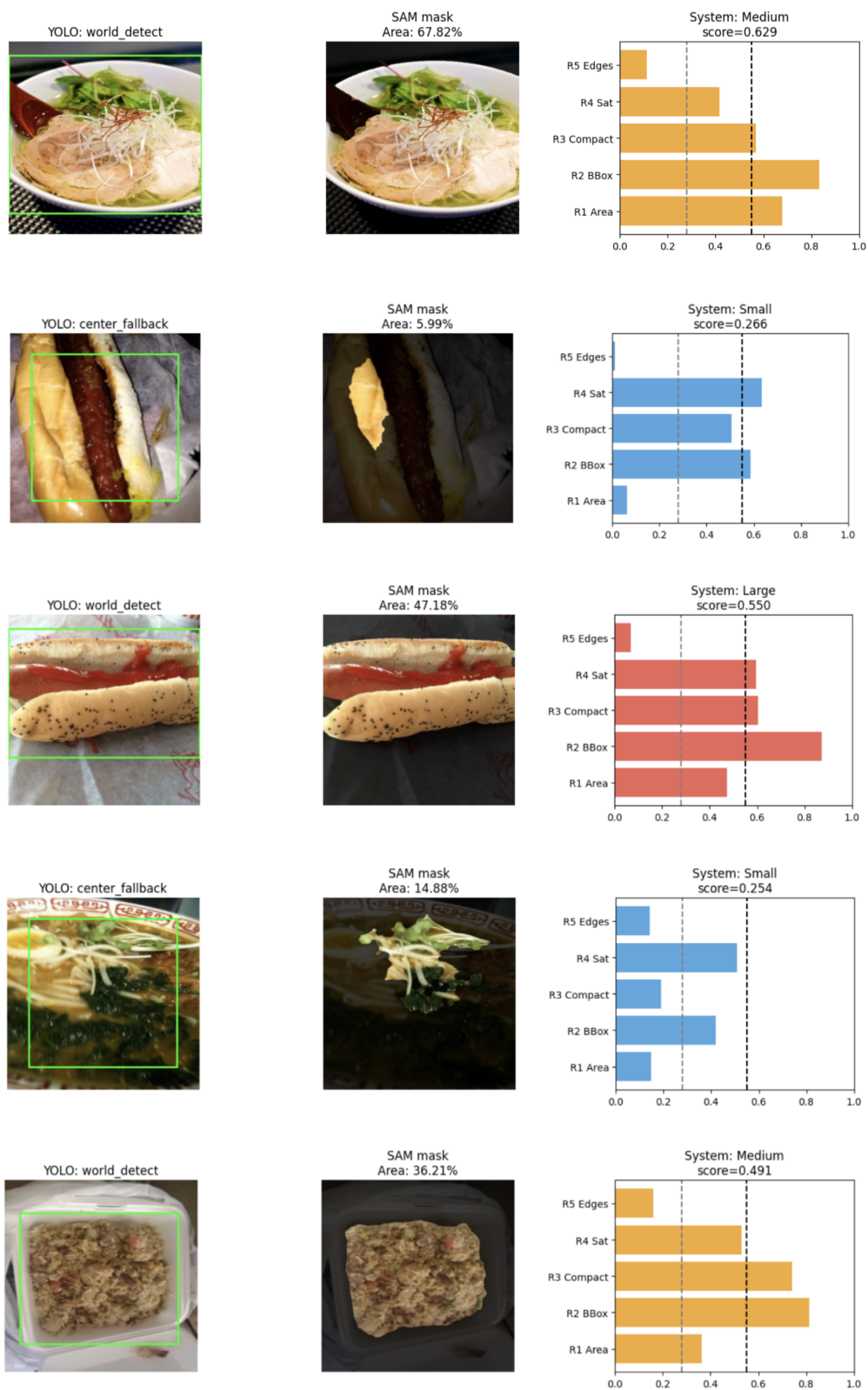
kể. Điều này làm hệ thống dễ dự đoán sai mức khẩu phần giữa các nhãn Small/Medium/Large mặc dù lượng thức ăn thực tế không thay đổi.

Ngoài ra, góc chụp khác nhau cũng ảnh hưởng mạnh đến diện tích biểu kiến của món ăn. Các ảnh chụp nghiêng hoặc chụp từ bên hông có thể làm món ăn trông lớn hơn hoặc nhỏ hơn so với ảnh chụp từ trên xuống, dẫn đến sai lệch trong quá trình ước lượng khẩu phần.

Một hạn chế khác đến từ bước phát hiện đối tượng bằng YOLO/YOLO-World. Trong một số trường hợp, mô hình detect sai vùng món ăn hoặc nhận diện nhầm sang các vật thể khác trong ảnh. Khi bounding box không bao phủ đúng món ăn, các bước xử lý tiếp theo sẽ bị ảnh hưởng dây chuyền, làm giảm độ chính xác cuối cùng.

Bên cạnh đó, bước segmentation vẫn chưa thật sự ổn định đối với các ảnh có nền phức tạp hoặc nhiều món ăn xuất hiện cùng lúc. Mô hình có thể segment lẫn sang đĩa, bàn ăn, nền ảnh hoặc các món ăn khác nằm gần nhau. Điều này làm diện tích món ăn được tính toán không phản ánh đúng kích thước thực tế của khẩu phần.

Hệ thống cũng gặp khó khăn với các món có hình dạng không cố định hoặc cách trình bày đa dạng. Ví dụ, cùng một loại thức ăn nhưng được bày trong tô, đĩa hoặc hộp khác nhau sẽ tạo ra sự khác biệt lớn về hình dạng và vùng chiếm ảnh.



Hình 3.8: Một số kết quả dự đoán khẩu phần của hệ thống

3.3.3 Phân tích giới hạn và hướng phát triển cho ước lượng khẩu phần

Pipeline hiện tại không đo khẩu phần thật – nó đo *cách thức ăn được trình bày trong ảnh*. Để xây dựng hệ thống ước lượng khẩu phần thực sự có giá trị ứng dụng, cần:

1. **Dataset có nhãn khẩu phần:** Ví dụ *Nutrition5k* (Google, 2021) gồm 5.000 đĩa thức ăn với RGB, depth map và nhãn khối lượng (gram) và calorie cho từng thành phần.
2. **Thông tin độ sâu (depth map):** Cho phép ước lượng thể tích thực của món ăn.
3. **Vật tham chiếu kích thước:** Đĩa chuẩn đường kính 26–28 cm giúp tính tỷ lệ pixel-to-cm.

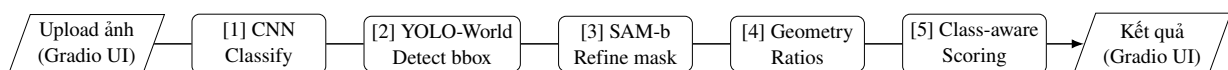
Pipeline lý tưởng cho bài toán ước lượng khẩu phần đầy đủ:

Ảnh RGB + Depth map $\xrightarrow{\text{Segment}}$ Thể tích $\xrightarrow{\times \text{mật độ}}$ Khối lượng (gram) $\xrightarrow{\text{USDA table}}$ Calorie

3.4 Xây dựng ứng dụng giao diện đồ họa (GUI)

3.4.1 Tổng quan kiến trúc ứng dụng

Nhóm xây dựng ứng dụng web GUI sử dụng thư viện **Gradio**, tích hợp toàn bộ pipeline thành một hệ thống trọn gói. Kiến trúc tổng quan:



Hình 3.9: Kiến trúc pipeline tích hợp trong ứng dụng GUI

3.4.2 Mô tả giao diện

Giao diện được chia thành hai cột chính:

Cột trái – Input:

- Upload ảnh từ file, webcam, hoặc clipboard.
- Nút *Analyze* để kích hoạt pipeline.
- Ảnh mẫu (examples) để demo nhanh.

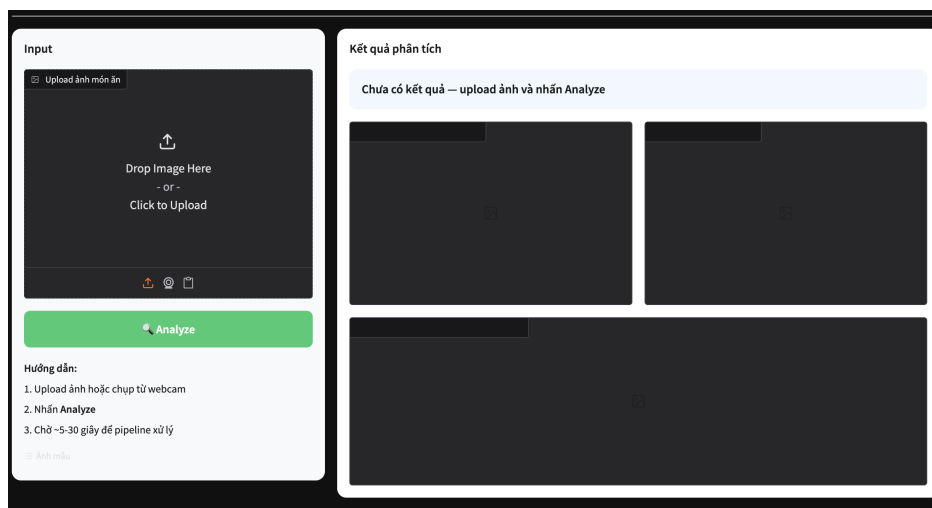
Cột phải – Output (hiển thị đồng thời sau khi phân tích):

- **Panel 1:** Ảnh gốc với YOLO bounding box (màu xanh lá) và nhãn CNN kèm confidence.

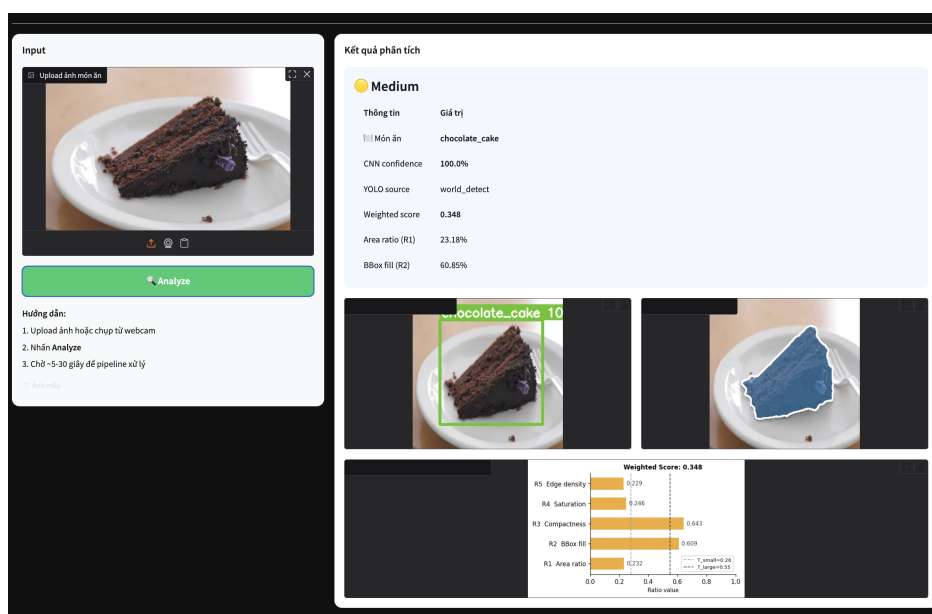
- **Panel 2:** Ảnh với SAM mask overlay (màu xanh lam bán trong suốt) và contour trắng.
- **Panel 3:** Biểu đồ horizontal bar chart 5 ratio kèm weighted score và ngưỡng phân loại.
- **Panel 4:** Bảng tóm tắt: tên món, confidence CNN, nguồn YOLO detect, score, và nhãn khẩu phần $\in \{\text{Small, Medium, Large}\}$.

3.4.3 Triển khai và demo

Ứng dụng được triển khai trên Kaggle Notebooks. Lệnh `demo.launch(share=True)` tạo link public dạng `*.gradio.live` có thể truy cập từ bất kỳ trình duyệt nào, phù hợp cho demo trong buổi báo cáo mà không cần cài đặt cục bộ.



Hình 3.10: Giao diện demo hệ thống nhận diện và ước lượng khẩu phần món ăn



Hình 3.11: Kết quả dự đoán món ăn và thông tin ước lượng

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Bài báo cáo đã thực hiện nghiên cứu bài toán phân loại món ăn trên tập con 15 lớp của bộ dữ liệu Food-101, từ các phương pháp cổ điển dựa trên đặc trưng thủ công đến các phương pháp học sâu hiện đại, đồng thời mở rộng sang bài toán ước lượng khẩu phần ăn ở mức thô và xây dựng ứng dụng giao diện. Cụ thể, nhóm đã triển khai và so sánh bốn hướng tiếp cận chính: Hướng 1 sử dụng đặc trưng HOG kết hợp SVM; Hướng 2 cải tiến bằng cách bổ sung Color Histogram trên không gian HSV; Hướng 3 áp dụng Transfer Learning với mạng EfficientNet-B0 trên ảnh gốc; và Hướng 4 đề xuất pipeline Segmentation-First kết hợp YOLO-World và Segment Anything Model để tách vùng thức ăn trước khi phân loại. Ngoài ra, nhóm còn xây dựng pipeline ước lượng khẩu phần dựa trên các đặc trưng hình học trích xuất từ mask phân đoạn và tích hợp toàn bộ hệ thống vào ứng dụng giao diện đồ họa.

Kết quả thực nghiệm cho thấy phương pháp học sâu vượt trội rõ rệt so với các phương pháp cổ điển. Trong khi các hướng tiếp cận thủ công chỉ đạt hiệu năng ở mức khiêm tốn do hạn chế trong khả năng biểu diễn đặc trưng, thì mô hình EfficientNet-B0 trên ảnh gốc cho thấy khả năng học đặc trưng phân cấp mạnh mẽ, mang lại kết quả phân loại cao và ổn định trên hầu hết các lớp. Pipeline Segmentation-First tuy chưa đạt hiệu năng bằng cách tiếp cận trên ảnh gốc do chất lượng phân đoạn còn hạn chế, nhưng đã chứng minh tính khả thi của hướng tiếp cận tách vùng quan tâm trước khi phân loại, đồng thời cung cấp mask hữu ích cho bài toán ước lượng khẩu phần. Việc bổ sung thông tin màu sắc vào đặc trưng thủ công cũng cho thấy mức độ cải thiện nhất định ở một số lớp phụ thuộc màu, tuy nhiên vẫn chưa đủ để thu hẹp khoảng cách lớn với các phương pháp học sâu.

Về hướng phát triển, có thể mở rộng nghiên cứu theo một số hướng sau: Thứ nhất, cải thiện chất lượng phân đoạn bằng cách fine-tune YOLO-World hoặc kết hợp thêm các mô hình phân đoạn mạnh hơn, đồng thời thử nghiệm các chiến lược crop thông minh để giảm thiểu mất mát ngữ cảnh. Thứ hai, mở rộng số lượng lớp và quy mô dữ liệu để đánh giá khả năng tổng quát hóa của mô hình trên bài toán thực tế lớn hơn. Thứ ba, tích hợp thêm các mô hình ước lượng khẩu phần chính xác hơn bằng cách kết hợp thông tin chiều sâu hoặc sử dụng dữ liệu có nhãn thực tế về khối lượng. Cuối cùng, tối ưu hóa hệ thống để có thể triển khai trên thiết bị di động hoặc edge device, hướng tới ứng dụng thực tiễn trong việc hỗ trợ theo dõi dinh dưỡng cá nhân và tự động hóa trong dịch vụ ăn uống.

Tài liệu tham khảo

- [1] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. IEEE CVPR*, vol. 1, pp. 886–893, 2005.
- [2] L. Bossard, M. Guillaumin, and L. Van Gool, “Food-101 – Mining Discriminative Components with Random Forests,” in *Proc. ECCV*, pp. 446–461, 2014.
- [3] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [4] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [5] T. Ojala, M. Pietikäinen, and T. Mäenpää, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE TPAMI*, vol. 24, no. 7, pp. 971–987, 2002.
- [6] M. J. Swain and D. H. Ballard, “Color indexing,” *International Journal of Computer Vision*, vol. 7, no. 1, pp. 11–32, 1991.
- [7] M. Tan and Q. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proc. ICML*, pp. 6105–6114, 2019.
- [8] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *Proc. ICML*, pp. 448–456, 2015.
- [9] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, “How Does Batch Normalization Help Optimization?” in *Proc. NeurIPS*, pp. 2488–2498, 2018.
- [10] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for Activation Functions,” arXiv preprint arXiv:1710.05941, 2017.
- [11] M. Lin, Q. Chen, and S. Yan, “Network in Network,” arXiv preprint arXiv:1312.4400, 2013.
- [12] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

- [13] S. J. Pan and Q. Yang, “A Survey on Transfer Learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.
- [14] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How Transferable Are Features in Deep Neural Networks?” in *Proc. NeurIPS*, pp. 3320–3328, 2014.
- [15] O. Russakovsky *et al.*, “ImageNet Large Scale Visual Recognition Challenge,” *International Journal of Computer Vision*, vol. 115, pp. 211–252, 2015.
- [16] J. Howard and S. Ruder, “Universal Language Model Fine-tuning for Text Classification,” in *Proc. ACL*, pp. 328–339, 2018.
- [17] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. CVPR*, pp. 4510–4520, 2018.
- [18] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *Proc. CVPR*, pp. 7132–7141, 2018.
- [19] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, “Deep Networks with Stochastic Depth,” in *Proc. ECCV*, pp. 646–661, 2016.
- [20] P. Micikevicius *et al.*, “Mixed Precision Training,” in *Proc. ICLR*, 2018.
- [21] A. Kirillov *et al.*, “Segment Anything,” in *Proc. ICCV*, pp. 4015–4026, 2023.
- [22] T. Cheng *et al.*, “YOLO-World: Real-Time Open-Vocabulary Object Detection,” in *Proc. CVPR*, 2024.
- [23] J. Pryor *et al.*, “Nutrition5k: Towards Automatic Nutritional Understanding of Generic Food,” in *Proc. CVPR*, pp. 8903–8911, 2021.